

SynthEval Studio and SynthEval Cloud: Quality-Gated Synthetic Data Generation for Regulated Industries

One filtering, masking, and evaluation methodology, two complementary
generation substrates engineered for cost and throughput

Patrick Hill, M.S. — Founder & Principal Engineer, PBH Applied Systems LLC

June 2026

Abstract

Published by PBH Applied Systems LLC, Oklahoma City. Contact: patrick@pbhapplyedsystems.com
| <https://pbhapplyedsystems.com>

Contents

1	Executive Summary	3
2	The Synthetic-Data Quality Problem	4
2.1	The Legal Exposure of Web-Scraped Training Data	5
2.2	The Regulatory Wall	7
2.3	The Financial Exposure	8
2.4	Two Substrates, One Methodology	10
2.5	The Layer That Is Missing	13
3	Architecture Overview	15
3.1	Shared Architecture and the Two Substrates	15
3.2	SynthEval Studio: Top-Level Layout	15
3.3	SynthEval Cloud: Top-Level Layout	16
3.4	The Core Engine	19
3.5	Data-Asset Library	20
3.6	Runtime Artifact Tree	20
3.7	Interface Surfaces	21
3.8	Operational Tooling	21
3.9	Generation and Embedding Models	22
3.10	Operational Parameters	23
3.11	Test Coverage	23
3.12	Runtime Flow	24
4	The Hallucination Gate	25
4.1	Hallucination as Instruction-Output Drift	25

4.2	The Dual-Signal Decision Rule	26
4.3	Calibrating the Threshold	26
4.4	Production Validation	27
4.5	Substrate Independence of the Gate	29
5	The Compliance Architecture	30
5.1	Compliance as a Gate, Not a Disclaimer	30
5.2	Layered PII Masking	30
5.3	Severity-Tiered Keyword Filters	31
5.4	Placeholder Fidelity	32
5.5	The Three-Category Acceptance Model	32
5.6	Independent Leak Auditing	33
5.7	Production Compliance Results	33
6	Throughput, Economics, and Quantization Format Selection	35
6.1	The Problem: Local Throughput Fails Time-Bound SLAs	36
6.2	Model Selection	36
6.3	Quantization Format Selection	36
6.3.1	Side-by-Side Results	37
6.3.2	GGUF — Fast but Quality-Capped	37
6.3.3	BitsandBytes — Parity but Slow and Costly	37
6.3.4	AWQ-vLLM — Breaking the Tradeoff	37
6.4	Batch-Size Optimization (AWQ-vLLM)	38
6.5	Vertical Scaling: Larger GPU Tiers	38
6.6	Horizontal Scaling: Concurrent Containers	40
6.7	Cost Model (Triangulated)	41
6.8	The Operational Case, Stated Plainly	41
7	The Evaluation Pipeline and the Attested Report	42
7.1	Evaluation as a Deliverable, Not a Diagnostic	42
7.2	The Five Metric Families	43
7.3	Reconciliation: Guarding Against Silent Loss	44
7.4	A Worked Report	44
7.5	When the Pipeline Caught a Real Defect	45
8	Results	46
8.1	The Three Runs	46
8.2	The Headline Finding: Quality Parity Across Substrates	47
8.3	Yield, Per-Domain Behavior, and the Compliance Trajectory	48
8.4	What the Results Establish	49
9	Economic Implications	50
9.1	The Development Economics: Converging on Quality, Not Budget	50
9.2	The Production Decision Framework	51
9.3	Owned Infrastructure Versus Metered Rental	52
9.4	The Value of Attestation	53
10	Limitations	53
10.1	Pattern-Based Compliance Cannot Be Proven Complete	54

10.2	Threshold Calibration Is Empirical, Not Adaptive	54
10.3	Substrate Bounds	55
10.4	The Evaluation-Layer Bias Metric Over-Reports in Regulated Domains .	55
10.5	The Bounds of the Evidence	56
11	Appendix	56
11.1	Configuration Reference	56
11.2	Run Inventory	57
11.3	Environment Specification — Local Substrate (SynthEval Studio)	58
11.4	Environment Specification — Cloud Substrate (SynthEval Cloud)	59
11.5	Glossary	59
12	References	61

1 Executive Summary

Synthetic training data has become a standard methodology at the frontier of model development, but the public synthetic-data ecosystem has not matured with it. Datasets are published with little documentation of how they were generated, what was filtered out, or what compliance posture they carry, at exactly the moment three forces have made that opacity untenable for commercial use. Copyright litigation has established that the provenance of training data is a liability, not a settled question. Privacy regulation — GDPR, HIPAA, CCPA — has made scraped personal data a source of unbounded and architecturally irreversible exposure. And the per-token economics of frontier generation APIs make the iterative development of a quality-attested dataset expensive enough to discourage it. Together these define a clear need: synthetic data produced under a controlled, documented, compliance-audited methodology, with quality and provenance that survive an audit.

This paper documents a methodology that meets that need, implemented as two products. A single quality-gated pipeline — semantic hallucination filtering, regulation-specific compliance masking, automated evaluation, and independent audit — is deployed across two complementary generation substrates. **SynthEval Studio** runs the methodology locally on a single consumer GPU, where near-zero marginal cost makes it the substrate for development and low-volume or flexible-timeline production. **SynthEval Cloud** runs the identical methodology with generation offloaded to a cloud GPU service, where higher throughput makes it the substrate for high-volume delivery under deadline. The two are not competing alternatives but complementary deployments of one methodology; the choice between them is an operational decision driven by volume and timeline, not by any difference in the data they produce. Frontier generation APIs are treated throughout as a neutral cost-and-capability baseline rather than an adversary: they generate raw tokens, while the value this paper documents is the production layer that turns generated tokens into an attested, compliant deliverable.

The central empirical finding is that the methodology produces equivalent datasets on both substrates. Across three production runs, lexical diversity (MATTR) held near 0.77 and instruction-output fidelity held near 0.73 whether generation ran lo-

cally or in the cloud — quality parity to within half a percentage point — because the quality-determining components run identically and locally regardless of where tokens are generated. On that parity, the cloud substrate delivers roughly thirteen times the local throughput at a measured cost near \$0.45 per thousand records, and local generation runs at roughly 26 to 52 times the cost efficiency of mid-tier and frontier APIs respectively. The hallucination gate is a dual-signal design that combines an embedding-similarity threshold with a keyword safety net, achieving precision against off-topic drift without discarding terse but on-topic domain outputs. The compliance architecture, after iterative hardening, reduced the residual personal-data leak rate in the delivered dataset to zero, measured by an independent scanner deliberately broadened to over-report rather than under-report — a result presented as measured and independently audited, never as guaranteed. The generation model and quantization format were both selected on measured evaluation performance, not convenience: a 12-billion-parameter model that outperformed a nearly three-times-larger alternative is what makes the cost-efficient substrate possible.

The methodology’s value, ultimately, is not the savings on token generation but the cost and risk it removes downstream. A regulated buyer receives a dataset accompanied by a machine-readable report covering rejection reasons, scale-invariant diversity, placeholder coverage, an independently audited leak rate, and a reconciliation of the run’s own counts — the verification work the buyer would otherwise have to perform or absorb as risk. The paper states its limitations as plainly as its results: pattern-based compliance cannot be proven complete, the thresholds are calibrated rather than self-adjusting, the larger-GPU and higher-concurrency economics are projections from measured single-substrate behavior, and the production runs are demonstrated at the thousand-record scale. What the evidence establishes is the core claim: one quality-gated methodology, deployed across two complementary substrates, produces attested, compliant, equivalent datasets at a cost and throughput that make defensible synthetic data practical for regulated industries.

2 The Synthetic-Data Quality Problem

Synthetic training data has moved from research curiosity to operating standard. The transition is most visible at the frontier: OpenAI, Google, Meta, and Anthropic all use synthetic or model-generated data in training, fine-tuning, post-training, alignment, or safety-evaluation workflows for major model systems (Anthropic, 2026; Gemini Team, Google, 2025; Guan et al., 2024; Meta AI, 2024; Wiggers, 2024). NVIDIA — the hardware vendor whose GPUs train every major model — has gone further and released Nemotron-4 340B, a model family explicitly designed for synthetic data generation, alongside a disclosure that more than 98% of the data used in its own model alignment process is synthetically generated (Adler et al., 2024). The category is no longer experimental. It is the default modern training methodology.

Yet the public synthetic-data ecosystem does not reflect this maturity. Datasets are published with minimal documentation of how they were generated, what was filtered out, what compliance posture they assume, or what quality guarantees they carry. A buyer evaluating a synthetic dataset for use in a regulated commercial setting — clinical NLP, legal-tech, financial services — typically finds a HuggingFace card with

row counts, a license, and little else. The generation pipeline is opaque. The rejection criteria are unstated. The compliance coverage is unaudited.

This gap exists because three forces have converged simultaneously, and the open ecosystem has not yet caught up to any of them: a wave of copyright litigation against the use of scraped web data for model training; a tightening regulatory environment in which scraped personal data carries unbounded legal exposure; and a cost structure for synthetic data generation in which the per-token rates of frontier generation APIs make the iterative development of a quality-attested dataset expensive enough to discourage it. Each of these is independently sufficient to motivate the adoption of compliance-documented, quality-gated synthetic data produced under a controlled methodology. Taken together, they define the market that the present work addresses.

The work documented in this paper addresses that market through a single quality-gated methodology — hallucination filtering, regulation-specific compliance masking, automated evaluation, and independent audit — deployed across two complementary generation substrates engineered for different operational constraints. SynthEval Studio runs the methodology on a local consumer-GPU workstation, where near-zero marginal cost makes it the natural substrate for development and for low-volume or flexible-timeline production. SynthEval Cloud runs the identical methodology on cloud GPU infrastructure, where higher throughput makes it the natural substrate for high-volume production under delivery deadlines that local generation cannot meet. The two substrates are not competing alternatives; they are complementary deployments of one methodology, and the choice between them is an operational decision driven by the volume and timeline of a given engagement. Frontier generation APIs are referenced throughout this paper as a cost and capability baseline, not as an adversary: they are a legitimate way to generate tokens, but they generate raw, un-gated, uncompliant output, and the layer this paper documents — the filtering, masking, evaluation, and audit that turn generated tokens into an attested deliverable — is absent from a raw API call regardless of which model serves it.

2.1 The Legal Exposure of Web-Scraped Training Data

For most of the last decade, the assumption underlying commercial model training was that web-scraped data was fair game under U.S. fair-use doctrine. That assumption is no longer safe to make. The legal landscape has shifted materially over the past eighteen months, and any organization training models on uncurated scraped data now operates under a substantial and growing liability footprint.

The most consequential development is *Bartz et al. v. Anthropic*. In June 2025, the U.S. District Court for the Northern District of California issued a split summary judgment: training on legally acquired books was held to be fair use, but downloading and storing copies from shadow libraries was not (Authors Guild, 2025). The piracy ruling led directly to a \$1.5 billion settlement — the largest copyright settlement in U.S. history (Wolters Kluwer, 2025). The final fairness hearing was held on May 14, 2026, with claims filed against approximately 440,490 of the 482,460 works on the certified works list, a 91.3% claims rate (Authors Alliance, 2026). The settlement establishes a per-title compensation framework of roughly \$3,000 per work and sets the operating

precedent that AI developers can be held liable for the *provenance* of their training data, not merely for whether the training itself qualifies as transformative use.

The implication for downstream users is direct. Plaintiff firms have begun developing what one commentator has termed the “Shadow Library Strategy”: skip the unresolved question of whether AI training is fair use, and litigate the piracy angle instead (Wolters Kluwer, 2025). Most new book-related copyright lawsuits against AI companies — including ongoing claims against Meta in *Kadrey v. Meta Platforms* — now allege downloads from shadow libraries (Norton Rose Fulbright, 2026). Any organization that fine-tunes a foundation model on a corpus of indeterminate provenance inherits this exposure.

In parallel, discovery practice in *In re: OpenAI, Inc. Copyright Infringement Litigation* has established that AI companies cannot keep their training logs and outputs private. On January 5, 2026, U.S. District Judge Sidney Stein affirmed an order compelling OpenAI to produce its entire 20-million-log sample, not a curated subset (National Law Review, 2026). On March 9, 2026, the court further compelled production of an additional 78 million and 10 million logs (Norton Rose Fulbright, 2026). The “Stein Standard” — that broad discovery prevails over developer-controlled search restrictions when copyright infringement is alleged — is now operative precedent. Every prompt-response pair preserved in an organization’s training infrastructure is potentially discoverable evidence.

The picture is not entirely one-directional. The UK High Court’s November 2025 decision in *Getty Images (US) Inc. v. Stability AI Limited* held that AI model weights themselves are not “infringing copies” under UK law, because the weights do not store or reproduce the underlying training images (Latham & Watkins, 2025; Mayer Brown, 2025). The ruling provided a narrow win for AI developers on the secondary-infringement theory. However, the *Getty* decision turned heavily on the factual concession that training had occurred outside the United Kingdom, and Getty had withdrawn its primary infringement claims before trial (Ropes & Gray, 2026). The case did not resolve whether training itself constitutes infringement — only that downstream distribution of a trained model is not, on these facts, a secondary infringement event. European jurisprudence has moved in the opposite direction: in November 2025, the Munich Regional Court held in *GEMA v. OpenAI* that training GPT-4 on copyrighted German song lyrics violated German copyright law, among the first major European rulings against an AI developer on training-data grounds (AI Lawsuit Tracker, 2026).

The practical reality for any organization fine-tuning models for commercial deployment is that the legal status of web-scraped training data is contested, jurisdictionally fragmented, and trending toward greater rather than lesser exposure. Indemnification clauses in vendor agreements increasingly reflect this — buyers now ask whether training data provenance can be documented, and a vendor who answers “we scraped the web” is at an active commercial disadvantage. Synthetic data generated under controlled provenance, with the generating model identified and the generation process documented, eliminates this exposure entirely. There is no shadow library to be discovered, no third-party rightsholder to assert a claim, no unresolved fair-use question to litigate. The data was generated, by a known model, under conditions the publisher can attest to.

2.2 The Regulatory Wall

Copyright is one source of training-data risk. Privacy regulation is the other, and for organizations operating in regulated industries it is the more immediately binding constraint.

Under the EU General Data Protection Regulation (GDPR), Article 5 imposes the principles of data minimization, purpose limitation, and storage limitation; Article 6 requires a valid lawful basis — consent, contract, legal obligation, vital interests, public task, or legitimate interest — for any processing of personal data (European Parliament & Council of the European Union, 2016). Web-scraped data for AI training struggles against every one of these requirements simultaneously. Consent is practically impossible to obtain from the unidentified rightsholders in a scraped corpus (Berberich, 2025). Purpose limitation is breached when data collected for one purpose is repurposed for model training without a new lawful basis. The European Data Protection Board’s December 2024 Opinion 28/2024 and the French CNIL’s June 2025 guidance both acknowledge that legitimate interest under Article 6(1)(f) can in principle support AI training, but only after a three-step assessment that most web-scraping operations cannot satisfy (Skadden, 2025; Orrick, 2025).

The architectural problem with web-scraped training data under GDPR is more fundamental than the lawful-basis question. Once personal data has been incorporated into model weights, it cannot be selectively removed. Article 17 — the right to erasure — assumes that personal data exists in a structured store from which it can be deleted. A neural network is not such a store. A June 2025 University of Tübingen study established that large language models memorize between 0.1% and 10% of training data verbatim (PPC Land, 2026), which means scraped personal data persists in deployed models in a form that may be retrievable but cannot be erased without retraining. This is not a compliance gap that can be closed with policy changes. It is a structural incompatibility between the architecture of foundation models and the rights regime under which European personal data is governed.

Enforcement has been escalating. Italy’s data protection authority, the Garante, issued a €15 million fine against OpenAI in December 2024 and banned DeepSeek from operating in Italy in January 2025 (PPC Land, 2026). It is worth noting that an Italian court in Rome annulled the penalty entirely in March 2026, ruling in favor of OpenAI (Wilson Sonsini Goodrich & Rosati, 2026). The Irish Data Protection Commission obtained X’s voluntary suspension of EU user data processing for AI training in August 2024 (Norton Rose Fulbright, 2025). Clearview AI has received fines ranging from €250,000 to €30.5 million across France, Greece, Italy, the Netherlands, and Sweden, with several authorities ordering deletion of scraped datasets (IAPP, 2025). The European Commission’s November 2025 Digital Omnibus initiative proposes amendments that would explicitly establish AI training as a legitimate interest under a new Article 88c, but Estonia, France, Austria, and Slovenia have formally opposed the changes, and the outcome remains politically contested (PPC Land, 2026).

The U.S. healthcare context is more constrained still. The Health Insurance Portability and Accountability Act (HIPAA) Privacy Rule prohibits use or disclosure of protected health information except under specific authorizations, and de-identification under the Safe Harbor method requires removal of 18 specified identifier categories.

Any organization training a clinical NLP model on scraped or repurposed patient data without express HIPAA authorization is operating outside the rule. The California Consumer Privacy Act (CCPA), as amended by the California Privacy Rights Act (CPRA), adds opt-out and deletion-right requirements that are similarly incompatible with the architecture of frozen training corpora.

The cumulative effect is that compliance is no longer a feature that can be bolted onto a model after training. It is a constraint that determines whether the model can be lawfully trained at all in the relevant jurisdiction. A synthetic dataset, generated by a known model under documented controls, containing no real personal data, and accompanied by an explicit compliance coverage report, sidesteps the entire regulatory wall. The data subjects of GDPR, HIPAA, and CCPA are real people whose data was processed; synthetic data has no data subjects. This is not a workaround. It is the only path that survives audit.

2.3 The Financial Exposure

The third force is economic, and it is the one most easily quantified. The most direct way to generate synthetic data without building infrastructure is to call a frontier generation API, which is priced per million tokens and billed separately for input and output. These rates establish the baseline against which any owned-infrastructure approach is measured. As of June 2026, OpenAI’s flagship GPT-5.5 is priced at \$2.50 per million input tokens and \$15.00 per million output tokens, with the more affordable GPT-5.4 at \$1.25 input and \$7.50 output (OpenAI, 2026). Anthropic’s Claude Opus 4.7 is priced at \$2.50 per million input tokens and \$12.50 per million output tokens, with the mid-tier Claude Sonnet 4.6 at \$1.50 input and \$7.50 output (Anthropic, 2026). Output tokens are priced five to six times higher than input tokens across the current generation of frontier models, and instruction-tuning datasets generate substantial volumes of both: each record consists of an input prompt that is billed as input and a generated output that is billed as output. Measured against the reference production run documented in Section 8, a representative record in this pipeline carries a median input length of 65 tokens and a mean output length of 425 tokens.

These rates are reasonable for what they provide — on-demand access to a frontier model with no infrastructure to operate — but they compound under the workflow that produces a defensible dataset, and the raw output they return still requires the entire filtering, masking, evaluation, and audit layer documented in this paper before it is commercially deliverable. A single 100,000-record dataset generated against GPT-5.5 costs approximately \$665; against Claude Opus 4.7, approximately \$557; against the mid-tier GPT-5.4 and Claude Sonnet 4.6, approximately \$332 and \$334 respectively (Table 2.1). Scaled to one million records, those figures range from approximately \$3,300 to approximately \$6,650. More importantly, every figure recurs on every regeneration. The development of a defensible dataset is rarely a single pass — it includes threshold calibration, prompt-template iteration, placeholder-coverage expansion, regeneration after schema changes, and re-runs after rejection-metric review. Under per-token API pricing, every one of those iterations is a billable event, and the meter does not stop between them. This is the cost structure that makes owned generation infrastructure worth building.

The alternative is local generation on consumer hardware. A single NVIDIA RTX 4090 — current-generation, approximately \$1,800 retail — runs a 4-bit quantized generation model and a 4-bit quantized embedding model simultaneously on a residential workstation, and the marginal cost of operation is electricity. Drawing on a representative residential utility bill from Oklahoma City covering a 32-day billing period at 990 kWh of consumption (Oklahoma Gas & Electric, 2026), the blended all-in residential rate is approximately \$0.134 per kWh, inclusive of fuel cost adjustment, franchise fee, sales tax, and the Winter Event Securitization charge applicable to Oklahoma customers.

The system’s power draw during generation was measured directly. GPU power averaged 144.27 watts across 3,896 telemetry samples collected at five-second intervals during active generation — approximately 32% of the RTX 4090’s 450-watt total graphics power, confirming that 4-bit quantized inference on this hardware is memory-bandwidth-bound rather than compute-bound (PBH Applied Systems LLC, 2026). Adding an estimated 38 watts for the host CPU at observed utilization and approximately 60 watts for the remaining workstation components yields a total system draw of approximately 242 watts, or a cost of operation of approximately \$0.032 per hour.

At the pipeline’s measured production throughput of approximately 251 accepted records per hour, a single 1,000-record production run completes in approximately 3.98 hours, consumes approximately 0.96 kWh, and costs approximately \$0.13 in electricity — under one-tenth of one percent of the workstation’s monthly residential power consumption. Larger deliverables are assembled by accumulating the per-run output of multiple production runs. A 100,000-record dataset costs approximately \$12.90 in electricity; a 1,000,000-record dataset, approximately \$129. Measured against current API rates and the same record token profile, local generation runs at approximately 26 times the cost efficiency of mid-tier APIs (GPT-5.4, Claude Sonnet 4.6) and approximately 43 to 52 times the cost efficiency of frontier APIs (Claude Opus 4.7, GPT-5.5), as shown in Table 2.1.

Table 2.1. *Marginal cost comparison for synthetic instruction-tuning dataset generation. Local costs reflect measured electricity consumption only and exclude amortized hardware. Cloud costs reflect published API rates as of June 2026, computed against the reference run’s measured token profile of 65 input and 425 output tokens per record.*

Dataset size	Local (RTX 4090)	GPT-5.4 (mid)	Sonnet 4.6 (mid)	Opus 4.7 (frontier)	GPT-5.5 (frontier)
1,000 records	\$0.13	\$3.32	\$3.34	\$5.57	\$6.65
10,000 records	\$1.29	\$33.24	\$33.41	\$55.68	\$66.49
100,000 records	\$12.90	\$332.43	\$334.08	\$556.81	\$664.86
1,000,000 records	\$129.04	\$3,324.32	\$3,340.84	\$5,568.07	\$6,648.64

Hardware amortization is not included in these figures. The RTX 4090 is a one-time capital expense that amortizes across an effectively unlimited number of generation cycles, and Section 6 documents the full cost model including depreciation and per-token economics under load. The relevant point for the demand-creation argument is the structure rather than the absolute savings: cloud generation imposes a linear per-token charge that scales without limit, while local generation imposes a fixed-cost workstation expense that amortizes across all subsequent runs. A publisher operating under cloud-API economics faces structural pressure to ship the first plausible dataset and stop, because every iteration is a billable event. A publisher operating on amortized local hardware can iterate until the rejection metrics, the calibration data, and the evaluation reports all converge on a defensible commercial claim. The two cost structures produce different products: one sized to the budget that generated it, the other sized to the quality standard the publisher chose to enforce.

2.4 Two Substrates, One Methodology

The cost analysis above establishes the marginal-cost floor. A complete account must also address time-to-delivery, because the cost and time axes are not symmetric, and an honest assessment of when each generation substrate is appropriate depends on naming both.

At the local pipeline’s measured throughput of 251 accepted records per hour, a 1,000-record dataset completes in approximately four hours, a 10,000-record dataset in approximately 1.7 days, a 100,000-record dataset in approximately 16.6 days, and a 1,000,000-record dataset in approximately 5.5 months of continuous operation. For development work and for low-volume or flexible-timeline production, this is not a constraint — it is the substrate on which cost-free iteration becomes possible. But for a buyer commissioning a 100,000-record regulated-industry dataset on a two-week deadline, sixteen days of local generation does not fit the timeline.

This is the constraint that SynthEval Cloud resolves. SynthEval Cloud is a second production substrate that runs the identical methodology — the same Mistral-Nemo generation model, the same Snowflake Arctic Embed evaluation, the same hallucination gate, the same regulation-specific PII masking, the same seven domains, and the same record-level acceptance logic — but moves inference to a Modal A10G GPU served by vLLM, while keeping evaluation local (PBH Applied Systems LLC, 2026). Where the local RTX 4090 runs deliberately memory-bandwidth-bound at approximately 32% of power draw to minimize cost, the cloud A10G runs deliberately compute-bound at sustained near-100% utilization and power to maximize speed, at an empirically determined optimal generation batch size of 128. The result is approximately 3,300 accepted records per hour — roughly thirteen times the throughput of the local substrate — at a triangulated all-in cost of approximately \$0.45 per 1,000 accepted records, with a marginal compute cost near \$0.00034 per record. A 1,000,000-record dataset that requires 5.5 months locally is delivered in approximately 12.6 days on the cloud substrate.

The three available substrates therefore occupy distinct positions, summarized in Table 2.2. Local generation is the development substrate and the low-volume production substrate, optimized for near-zero marginal cost. SynthEval Cloud is the high-volume,

tight-timeline production substrate, optimized for throughput while preserving every quality and compliance gate. A generic frontier API is faster still and requires no infrastructure, but it returns raw, unfiltered, non-compliant generations that the buyer must then filter, mask, and audit independently — which is precisely the problem this paper addresses.

Table 2.2. *The three generation substrates. Local and SynthEval Cloud both run the complete SynthEval methodology — hallucination gate, regulation-specific masking, evaluation, and audit. A generic frontier API provides none of these.*

Substrate	Cost per 1,000	Throughput	1M-record delivery	Quality & compliance gates
SynthEval Studio (local RTX 4090)	~\$0.13	~251 records/hr	~5.5 months	Full methodology
SynthEval Cloud (Modal A10G)	~\$0.45	~3,300 records/hr	~12.6 days	Full methodology, identical
Generic frontier API (e.g. GPT-5.5)	~\$6.65	very high	~18 hours	None — raw generations

The strategic consequence is that the substrate choice is a per-engagement decision driven by dataset size and delivery timeline, not a per-vendor commitment, and both substrates that run the full methodology are owned and operated by the same publisher. The methodology is the product. The substrate is an operational parameter. The cloud substrate’s record-level acceptance model — under which hard violations are rejected outright and never delivered, detected personal data is masked in place, and only recoverable placeholder-density shortfalls are regenerated under the same gates — is documented in Sections 5 and 7, and its independent post-run PII audit, in which a separate broader-pattern scanner corroborates the in-pipeline leak metric rather than sharing its logic, is the difference between privacy measured and privacy asserted.

The cloud substrate’s throughput is bounded by the GPU it runs on, and that boundary is not fixed. The A10G results above were measured to be compute-bound at the optimal batch size: GPU telemetry during production runs showed sustained utilization and power near 100% with memory held well below capacity (PBH Applied Systems LLC, 2026). A compute-bound workload scales with the compute and memory-bandwidth headroom of a larger card, which means the same methodology, unchanged, projects to substantially higher throughput on data-center GPUs. Using the more conservative of the compute-throughput and memory-bandwidth scaling ratios from published vendor specifications — the defensible floor rather than the optimistic ceiling, and assuming the batch size is re-tuned to the new card’s optimum — an NVIDIA A100 80GB is projected to deliver approximately 8,200 accepted records per hour and a one-million-record dataset in approximately five days, and an NVIDIA H100 80GB approximately 18,400 accepted records per hour and a one-million-record dataset in approximately 2.3 days (Table 2.3).

The cost behavior under this scaling is the decisive point for competitiveness. Modal’s hourly rate rises with GPU tier, but throughput rises faster, so the all-in cost per thousand accepted records falls as the substrate gets faster. Applying each GPU’s published Modal hourly rate, adjusted by the same cold-start and idle-tail overhead measured on the A10G run, against its projected throughput, the all-in cost per thousand records is approximately \$0.45 on the A10G, approximately \$0.41 on the A100, and approximately \$0.29 on the H100. A one-million-record dataset — fully gated, masked, evaluated, and audited — is therefore projected to cost approximately \$450 and take 12.6 days on the A10G, approximately \$412 and 5.1 days on the A100, or approximately \$290 and 2.3 days on the H100. The same one-million-record volume generated as raw, ungated tokens against a frontier API at the reference token profile costs approximately \$3,300 on the mid-tier models and approximately \$5,500 to \$6,500 on the flagship models (Table 2.1), and still requires the entire filtering, masking, evaluation, and audit layer applied afterward before it is deliverable. The projection therefore makes the competitive claim concrete: on an H100, the methodology delivers a complete, attested million-record dataset in roughly two days at under a tenth of the token cost of generating the same volume of unfiltered output on a frontier API. These are projections grounded in measured compute-bound behavior and published GPU pricing, not measured runs, and are presented as such.

Throughput scales on a second, independent axis as well. The figures above describe a single generation container; because the cloud substrate dispatches work to a serverless GPU service, the same methodology also scales horizontally by raising the number of concurrent containers, with aggregate throughput rising roughly in proportion to the container count and cost-per-record remaining approximately flat, since throughput and hourly burn scale together. Vertical scaling — a larger GPU per container — moves single-stream delivery speed toward the frontier-API range; horizontal scaling — more containers in parallel — closes the remaining gap to frontier-equivalent aggregate throughput, bounded in practice by the local evaluation stage, which must be parallelized to keep pace. Section 6 documents both scaling axes and the container-concurrency projection in full; the point here is only that the cloud substrate’s delivery speed is not fixed at the single-container rate, and that approaching frontier-equivalent throughput is a matter of provisioned concurrency rather than a different methodology.

Table 2.3. *Projected SynthEval Cloud cost and throughput across GPU tiers, for a one-million-record dataset. The A10G row is measured; the A100 and H100 rows are conservative projections using the lower of compute and memory-bandwidth scaling ratios from published specifications and each GPU’s published Modal hourly rate, assuming per-GPU batch-size re-tuning. All tiers run the identical methodology and deliver fully gated, masked, evaluated, and audited records. Frontier-API figures from Table 2.1 are shown for reference and represent raw, ungated token cost only.*

Substrate / tier	Status	Throughput	Cost per 1,000	1M- record cost	1M- record time
Modal A10G	measured	~3,300 records/hr	~\$0.45	~\$450	~12.6 days

Substrate / tier	Status	Throughput	Cost per 1,000	1M-record cost	1M-record time
Modal A100 80GB	projected	~8,200 records/hr	~\$0.41	~\$412	~5.1 days
Modal H100 80GB	projected	~18,400 records/hr	~\$0.29	~\$290	~2.3 days
Frontier API (mid-tier)	reference	very high	~\$3.30	~\$3,300	~hours, raw only
Frontier API (flagship)	reference	very high	~\$5.50-\$6.65	~\$5,500-\$6,650	~hours, raw only

Moving SynthEval Cloud to a larger GPU is a configuration change rather than a re-engineering effort, because the methodology layer is hardware-independent. In practical terms it requires only that the GPU type be changed in the Modal service declaration; that the generation batch size be re-tuned with the existing benchmark script, since the optimal batch grows with the card; that the model’s context-length and key-value-cache settings be widened to use the larger memory; and that the per-record cost be re-measured against the new GPU’s hourly rate. An optional higher-precision or FP8 model variant becomes available on these cards but is not required. Nothing in the hallucination gate, the masking architecture, the evaluation pipeline, or the audit changes — the same local Arctic Embed evaluation and the same acceptance logic run unmodified. The portability that lets the methodology move between a local workstation and the cloud is the same portability that lets it move between GPU tiers within the cloud, and Section 6 documents the batch-size optimization methodology that makes the re-tuning step a single repeatable benchmark.

2.5 The Layer That Is Missing

NVIDIA’s release of Nemotron-4 340B, together with its own disclosure that 98% of its alignment data is synthetic (Adler et al., 2024), and the parallel emergence of DeepSeek-R1 — an MIT-licensed open-source reasoning model trained on synthetic chains-of-thought generated by the model itself (DeepSeek-AI, 2025) — confirms that synthetic data is now the operating standard at the frontier. The category is validated. What is not validated, and what the open ecosystem has not yet built, is the production layer that sits between raw generation capacity and a commercially deployable, compliance-ready dataset deliverable.

That layer is a methodology, not a piece of hardware, and the central architectural claim of this paper is that the methodology is portable across generation substrates. The same four quality and compliance components are applied identically whether generation runs on a local consumer GPU or on cloud infrastructure; only the generation step beneath them changes. None of these four components is provided by a foundation model or a base generation pipeline on its own:

A hallucination filter calibrated to the specific embedding distribution of the production pipeline, not inherited from a prototype or a literature default. Section 4 docu-

ments the calibration of this filter for the present system.

A layered compliance filtration architecture that operates against named regulatory frameworks — GDPR, HIPAA, CCPA — combining tiered personal-information masking with severity-tiered keyword filters for bias mitigation and regulation-specific terminology rather than relying on a generic regex catalog, and validated by an independent post-run audit scanner that corroborates the in-pipeline leak metric rather than sharing its detection logic. Section 5 documents the masking architecture, the bias and regulatory keyword tiers, the regulation-specific tier definitions, the record-level acceptance model, and the audit examples that demonstrate coverage.

An automated evaluation pipeline that produces an attached, machine-readable report for every dataset shipped, covering rejection-reason breakdowns, scale-invariant diversity metrics, placeholder coverage statistics, and reconciliation between accepted counts, rejection logs, and run manifests. Section 7 documents the evaluation pipeline and presents a sample report.

A transparent residual-risk posture under which every delivered record has cleared the rejection gates and had detected personal data masked in place, while residual measured rates — such as the mild bias signal that falls below the rejection threshold — are reported on the accepted set rather than suppressed. This distinction between gate-passed and provably perfect is the difference between a defensible quality claim and an indefensible one, and Sections 5 and 10 document it directly.

Because the methodology is independent of the substrate beneath it, it can be deployed where the operational constraints of a given engagement dictate. SynthEval Studio runs it on a local consumer GPU at near-zero marginal cost, the substrate for development and for low-volume or flexible-timeline production. SynthEval Cloud runs it on cloud GPU infrastructure at roughly thirteen times the throughput, the substrate for high-volume delivery under deadlines local generation cannot meet. The gates, the masking, the evaluation, and the audit are identical on both. Section 6 documents the throughput, batch-size optimization, and economics of each substrate, and Section 3 documents the shared architecture that makes the methodology portable between them.

These components, taken together, define the gap that this paper addresses. NVIDIA legitimizes the category. DeepSeek demonstrates the methodology at the frontier. Open-weight foundation models from a small number of frontier developers provide the generation substrate on which any pipeline can be built, whether that substrate is a local workstation or a cloud GPU service — Section 3 identifies the specific model selected and the rationale for its selection. Frontier generation APIs remain available to anyone as a no-infrastructure generation option, at the rates documented in Section 2.3 and without any of the layer above. What turns generation capacity, on any substrate, into an attested deliverable is the methodology layer. The remainder of this paper documents that layer and the two substrates that carry it.

3 Architecture Overview

SynthEval is a single quality-gated methodology — instruction assembly, generation, semantic hallucination filtering, regulation-specific compliance masking, automated evaluation, and independent audit — implemented as a config-driven, modular Python pipeline and deployed across two complementary generation substrates. SynthEval Studio runs the methodology entirely on a local consumer-grade workstation centered on an NVIDIA RTX 4090, loading the generation and embedding models in-process. SynthEval Cloud runs the identical methodology with generation offloaded to a remote Modal A10G GPU served by vLLM, while retaining evaluation on the local workstation. The two substrates share the same architecture, the same data assets, the same quality gates, the same masking architecture, and the same evaluation pipeline; they differ only at the single boundary where generation tokens are produced. This section documents that shared architecture, identifies the points where the two substrates diverge, names the generation and embedding models, and describes the runtime flow through which an instruction-tuning record moves from request to attested output on either substrate.

3.1 Shared Architecture and the Two Substrates

The central architectural property of SynthEval is that the methodology is independent of the substrate that generates tokens. Both deployments are organized as a single Python package — `syntheval_studio` and `syntheval_cloud` respectively — nested under a repository root of the same name, with a core engine, a static data-asset library, a runtime-artifact tree, and an interface surface. In both, configuration is declarative, domain assets are version-controlled and static, runtime artifacts are written per-run and never modified after the fact, and executable code calls into configuration and assets but never the reverse. This separation is what makes either substrate auditable: every record produced can be traced deterministically to the configuration, templates, and code version that produced it.

The divergence between the two substrates is confined to one stage. In SynthEval Studio, the generation model is loaded into the workstation’s GPU memory and invoked in-process. In SynthEval Cloud, the generation step resolves a handle to a deployed Modal vLLM service and dispatches batches to it over the network, while the embedding model that drives the hallucination gate remains local. Every stage downstream of generation — the semantic similarity gate, the PII masker, the keyword tiers, the acceptance logic, the evaluation pipeline, and the audit — is byte-for-byte the same methodology on both substrates. The remainder of this section describes that shared methodology once and notes the substrate-specific divergences where they occur.

3.2 SynthEval Studio: Top-Level Layout

The SynthEval Studio repository is rooted at `syntheval_studio/` and divides cleanly into six top-level directories, each with a single architectural responsibility (Figure 3.1). The `core/` directory contains the engine — configuration management, generation orchestration, quality filtering, compliance masking, evaluation, and metrics computation. The `data/` directory contains all domain-specific assets: domain defi-

nitions, persona libraries, instruction templates, and topic ontologies, organized in parallel by domain. The `runs/` directory is the structured artifact tree where every invocation materializes its outputs, rejections, audit logs, and evaluation reports under a timestamped run identifier. The `interface/` directory exposes two entry points — a command-line interface and an HTTP API — both backed by the same orchestration logic. The `scripts/` directory holds operational utilities. The `tests/` directory contains the unit and integration suite. At the repository root, `config.yaml` is the master configuration file, `requirements.txt` is dependency-locked, `pyproject.toml` declares package metadata, and a Python virtual environment is provisioned under `synth_env/`.

Figure 3.1. *SynthEval Studio project structure, showing the core engine, data-asset library, runtime artifact tree, interface surfaces, operational scripts, and test suite together with their constituent modules. The configuration block identifies the generation and embedding models, the calibrated hallucination thresholds, the operational targets, and the automatic run-mode selection rules described in §3.10.*

The layout reflects a deliberate design principle: configuration data, generative assets, runtime artifacts, and executable code occupy separate directories with no cross-pollination. This is the separation that makes the pipeline auditable, and it is preserved unchanged in the cloud substrate.

A structural note worth flagging is that the repository root and the inner package share the name `syntheval_studio`, so module paths are doubled — `core/` sits at `syntheval_studio/syntheval_studio/core/` relative to the repository root. This is the source of a common path error when commands are issued from the wrong directory.

3.3 SynthEval Cloud: Top-Level Layout

The SynthEval Cloud repository mirrors the Studio layout, organized as a single Python package `syntheval_cloud` nested under a repository root of the same name (Figure 3.2). The `core/` engine carries the same sub-packages — `config/`, `generation/`, `masking/`, `quality/`, and `evaluation/` — and the `data/` library carries the same seven domains. The structural differences from Studio are three, and each follows from the substrate’s purpose. First, a `modal_app/` directory contains `vllm_server.py`, the deployed Modal A10G vLLM service that performs remote generation; this directory has no counterpart in Studio, where generation is in-process. Second, the `interface/` directory exposes a command-line interface only, with no HTTP API surface. Third, the `scripts/` directory carries two additional utilities specific to the cloud workflow — `benchmark_batch_size.py`, which determines the optimal generation batch size, and `audit_pii.py`, which performs an independent post-run PII leak audit — alongside the shared `mask_audit.py`, `clean_legacy_datasets.py`, and `report_summary.py`. The repository root holds `config.yaml` and a Python 3.10 virtual environment under `synthc_env/`.

Figure 3.2. *SynthEval Cloud project structure, rendered in the same format as Figure 3.1. The core engine, data-asset library, runtime artifact tree, and interface mirror SynthEval Studio. The `modal_app/vllm_server.py` service is the substrate-specific divergence: it performs remote generation on a Modal A10G GPU, replacing Studio’s in-process generation while leaving every downstream stage unchanged.*

A structural note worth flagging is that the repository root and the inner package

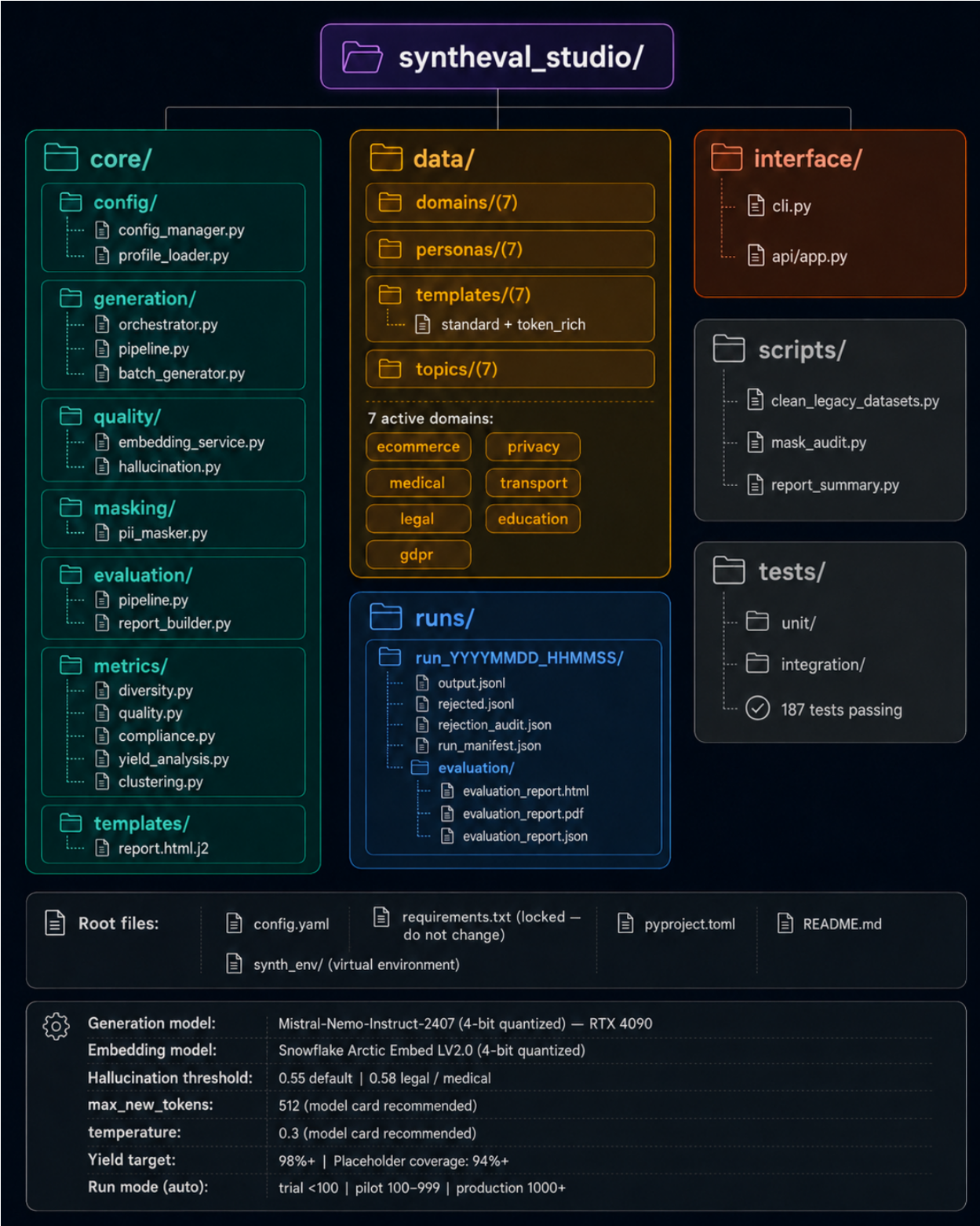


Figure 1: Figure 3.1 — SynthEval Studio project structure

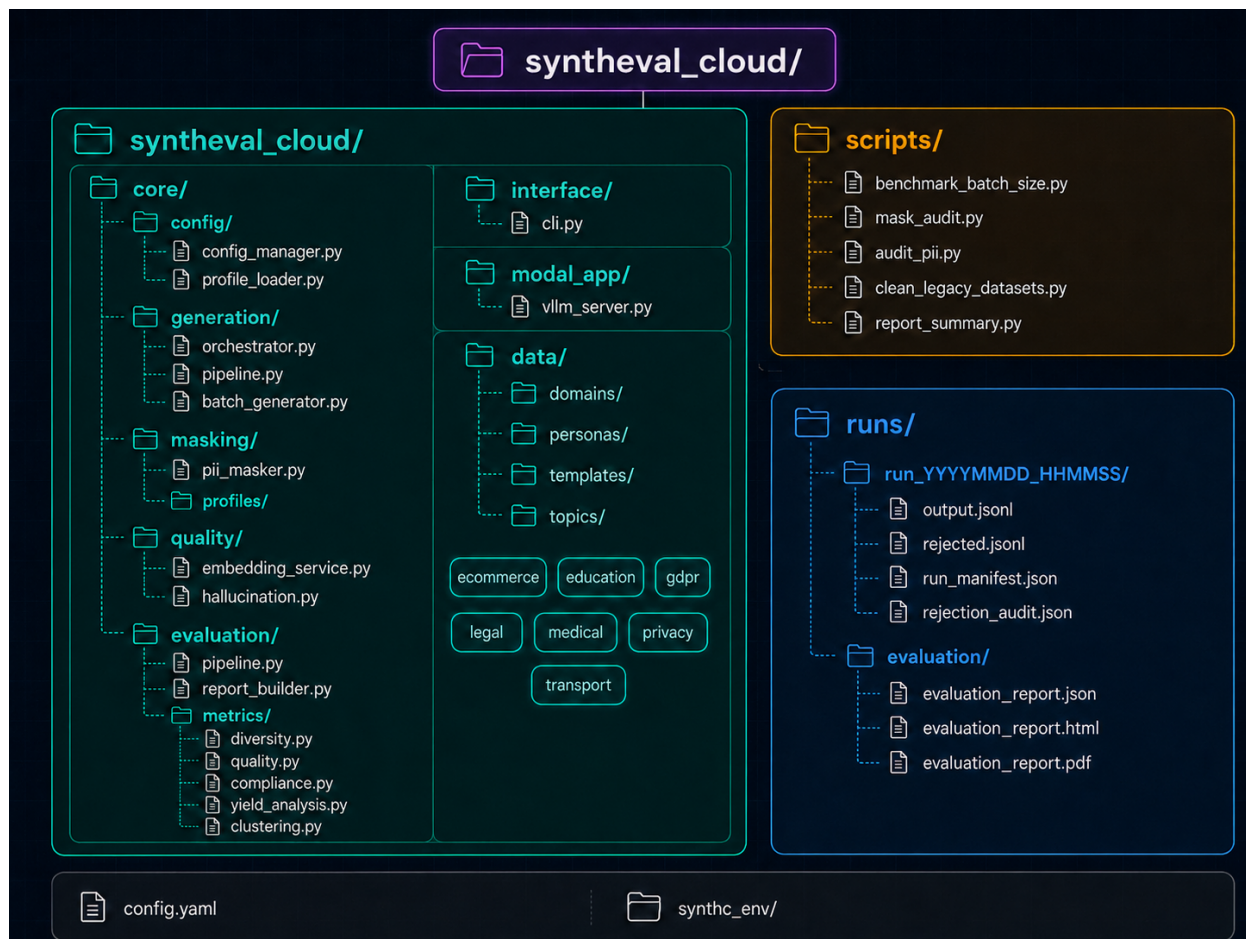


Figure 2: Figure 3.2 — Syntheval Cloud project structure

share the name `syntheval_cloud`, so deployment paths are doubled — the Modal service is deployed from `syntheval_cloud/modal_app/vllm_server.py` relative to the repository root. This is the source of a common path error when the deploy command is issued from the wrong directory.

3.4 The Core Engine

The `core/` directory contains the same functional sub-packages on both substrates, each corresponding to a stage of pipeline operation.

The `config/` sub-package resolves runtime parameters through a two-layer model. `config_manager.py` loads the master `config.yaml` and provides typed configuration accessors; `profile_loader.py` resolves a generation profile — a coherent bundle of model parameters, template selections, prompt-format directives, and quality-gate settings — against the domain selected for the current run. Each of the seven active domains carries its own profile, and the loader merges per-domain settings with the global configuration. This two-layer approach is what permits the legal and medical domains to operate with a tighter hallucination threshold than the general-purpose domains without branching code paths.

The `generation/` sub-package orchestrates the generative half of the pipeline through three modules, and this is the one sub-package where the substrates diverge internally. `orchestrator.py` is the entry point invoked by the interface layer and coordinates the full lifecycle of a generation request: profile resolution, the run loop, gating, retries, and — in the cloud substrate — per-batch checkpointing. `batch_generator.py` assembles instruction batches from the templated prompt libraries, pulling persona, topic, and template combinations and applying domain weighting to construct the persona-framed, template-expanded instructions that drive generation. `pipeline.py` is the inner loop. In SynthEval Studio it dispatches assembled instructions through the in-process generation model; in SynthEval Cloud it resolves a Modal vLLM handle and dispatches each batch to the remote engine via `generate_batch()`. In both cases it emits raw outputs paired with their originating instructions to the downstream quality stages. The interface and the contract of `pipeline.py` are identical across substrates; only the generation backend behind it changes.

The `quality/` sub-package is the first gate every generated output traverses, and it is identical on both substrates — including in the cloud deployment, where it runs locally. `embedding_service.py` wraps the Snowflake Arctic Embed L v2.0 model and exposes `batch_similarity()` for computing semantic embeddings of instruction-output pairs and their cosine similarity. `hallucination.py` implements the gate logic, comparing each pair’s cosine similarity against the calibrated threshold and routing the output either onward to the masking layer or to the rejection log. Section 4 documents the calibration of the threshold; the present section addresses only its architectural placement. That the embedding gate runs locally even when generation is remote is a deliberate design choice. It keeps the quality signal on hardware the operator controls and avoids shipping embeddings off the workstation.

The `masking/` sub-package contains `pii_masker.py`, the entry point into the layered compliance filtration architecture, together with a `profiles/` directory of per-domain allowed and forbidden placeholder sets. The masker applies PII pattern detection,

an always-mask floor, and fidelity checks against named regulatory frameworks — GDPR, HIPAA, and CCPA. The full unpacking of the masking architecture, including the bias-keyword tiers, the regulatory-keyword tiers, the regulation-specific detection logic, and the record-level acceptance model, is the subject of Section 5; the present section locates the masker in the pipeline flow.

The `evaluation/` sub-package operates after the generation phase has completed. `pipeline.py` drives the post-run evaluation against the accepted output set, and `report_builder.py` assembles the multi-format evaluation report — HTML, PDF, and machine-readable JSON. Nested beneath it, the `metrics/` directory contains the modules the evaluation pipeline composes: `diversity.py` measures lexical and semantic diversity; `quality.py` aggregates per-record quality signals; `compliance.py` computes the PII leak rate, echo rate, and GDPR surface rate; `yield_analysis.py` characterizes the acceptance-versus-rejection profile by reason category; and `clustering.py` performs semantic clustering of the output corpus to expose mode collapse, persona collapse, or topic underrepresentation that the unit-level filters would not catch. Evaluation is not a sampling step; the entire accepted corpus passes through the pipeline, and the resulting report is written to the run’s `evaluation/` subdirectory as a permanent artifact bound to that run’s output.

3.5 Data-Asset Library

The `data/` directory is the static asset library, and it is shared identically by both substrates. It is organized by parallel structure rather than by content type: four sub-directories — `domains/`, `personas/`, `templates/`, and `topics/` — each contain coordinated assets for the seven active domains: *ecommerce*, *privacy*, *medical*, *transport*, *legal*, *education*, and *gdpr*. Each domain carries its own domain definition, persona library, template set, and topic ontology, and these four assets together fully parameterize the generative behavior for that domain.

Adding a new domain is a static-file operation — the engineer adds the parallel files under `data/`, registers the domain in `config.yaml`, and the pipeline picks it up on the next invocation, on either substrate. No code change is required to add a domain. This is the configuration-driven discipline that lets the same data library serve both the local and cloud deployments without modification: the engine is general, the domains are data.

3.6 Runtime Artifact Tree

Every invocation of either substrate produces a complete, self-contained run record under `runs/`, written to a timestamped directory in the form `run_YYYYMMDD_HHMMSS/`. The directory structure is uniform across every run regardless of substrate, domain, or run size, and the contents constitute the audit substrate of the system.

Five artifacts are written to the run directory. `output.jsonl` contains the accepted records — every output that survived the hallucination gate, the PII masker, and the keyword tiers. `rejected.jsonl` contains every record that failed one or more gates, paired with its originating instruction so that rejected records remain interpretable in isolation. `rejection_audit.json` aggregates rejection counts by reason

category and provides the data substrate for the report’s rejection-breakdown visualizations. `run_manifest.json` records the run’s metadata in totality: the resolved profile, the model identifiers, the counts, the yield, the reconciliation between accepted and rejected tallies, and the provenance needed to reproduce the run. The `evaluation/` subdirectory contains the three report artifacts — `evaluation_report.json` as the canonical machine-readable representation, with `evaluation_report.html` and `evaluation_report.pdf` as rendered views of the same underlying data.

The substrates differ in one operational respect within this tree. SynthEval Studio accumulates accepted records and writes the output at run completion. SynthEval Cloud writes each batch’s accepted and rejected records to disk immediately, with a filesystem sync, as the batch finalizes. The consequence is crash resilience: an interrupted cloud run leaves its completed batches durably on disk — a failure at batch seven of ten leaves roughly 870 valid records recoverable and fully evaluable through the `evaluate` command — rather than discarding the work. The scope is partial-persistence recovery of the completed portion, not automatic resume. This difference reflects the cloud substrate’s longer wall-clock runs and its dependence on network RPCs, where the probability of an interruption over the course of a large run is non-trivial and the cost of losing it is higher.

The append-only discipline is otherwise identical and structurally enforced on both substrates: once a run completes, its artifacts are never rewritten, and reruns generate new timestamped directories. Every dataset published from either substrate therefore carries an immutable provenance trail back to the exact run, configuration, and rejection profile that produced it.

3.7 Interface Surfaces

SynthEval Studio exposes two entry points from its `interface/` directory: `cli.py`, a command-line interface for operational and scripted use, and `api/app.py`, an HTTP API surface for service-oriented consumption. Both are backed by the same orchestration logic, accept the same parameters, and write to the same `runs/` directory.

SynthEval Cloud exposes a command-line interface only, through `cli.py`, with `run`, `evaluate`, `clean`, and `info` commands. The absence of an HTTP API on the cloud substrate reflects its operational model: cloud runs are launched deliberately as batch jobs against the deployed Modal service rather than served as an on-demand request endpoint. The `evaluate` command is operationally significant on this substrate because it permits a partially completed run — for example, one recovered after an interruption — to be evaluated against whatever records were durably checkpointed, decoupling evaluation from generation.

3.8 Operational Tooling

Both substrates place operational utilities in a `scripts/` directory, outside the `core/` engine, reflecting the principle that tooling supporting the publishing workflow should not be carried inside the generative substrate. The shared utilities are `clean_legacy_datasets.py`, which normalizes datasets from earlier pipeline versions and removes echo contamination; `mask_audit.py`, which produces a masking demon-

stration report; and `report_summary.py`, which rolls a run’s evaluation data into a single-page summary.

SynthEval Cloud adds two utilities specific to its operating profile. `benchmark_batch_size.py` performs the batch-throughput sweep that determines the optimal generation batch size for the deployed engine — the methodology behind this is documented in Section 6. `audit_pii.py` performs an independent raw-PII leak audit using deliberately broader detection patterns than the in-pipeline masker, so that the leak metric is corroborated by a scanner that does not share the masker’s blind spots. This independent audit is the architectural expression of a hard-won lesson: a privacy metric that reuses the masker’s own patterns can report a clean result while a raw value is present, and a privacy metric that can under-report is worse than the leak it conceals. Section 5 documents the audit’s role in the compliance argument.

3.9 Generation and Embedding Models

Both substrates use the same two models: Mistral-Nemo-Instruct-2407 as the generation model and Snowflake Arctic Embed L v2.0 as the embedding model that drives the hallucination gate. Both models are open-weight under the Apache 2.0 license, and the choice of open-weight models is what makes the methodology portable across substrates and deployable without per-token licensing or API dependency. In both cases the models were quantized in-house by PBH Applied Systems for their respective serving environments rather than consumed as pre-quantized third-party builds, which is what allows the same source models to be matched precisely to each substrate’s hardware profile.

Mistral-Nemo-Instruct-2407 is the instruction-tuned variant of the 12-billion-parameter Mistral-Nemo model jointly developed by Mistral AI and NVIDIA and released in July 2024 under the Apache 2.0 license (Mistral AI, 2024; Mistral AI, 2025). The two substrates run the same model in two quantization-and-serving configurations. SynthEval Studio loads a 4-bit quantized build into the RTX 4090’s memory and invokes it in-process; at 4-bit quantization the model occupies approximately 7.8 GB of VRAM, leaving ample headroom for the embedding model to co-reside on the same 24 GB GPU. SynthEval Cloud serves an AWQ-quantized build of the same model on a Modal A10G GPU through vLLM, which provides paged key-value-cache management and concurrent batch processing. Two generation parameters are pinned to model-card-recommended values on both substrates: temperature is set to 0.3, the value Mistral AI advises for stable Nemo output, and the generation length is bounded by the production token budget.

Snowflake Arctic Embed L v2.0 is a 568-million-parameter retrieval-optimized embedding model released in December 2024 under the Apache 2.0 license (Snowflake, 2024; Yu et al., 2024). It is operationally decisive for three reasons: it is optimized for retrieval-style semantic similarity, which is exactly the task the hallucination gate requires; it supports dimensionality reduction with minimal quality loss, keeping the embedding service inexpensive; and at 4-bit quantization it occupies under one gigabyte of VRAM. Critically, the embedding model runs locally on the RTX 4090 in both substrates — even in SynthEval Cloud, where generation is remote. The quality signal stays on operator-controlled hardware regardless of where tokens are generated.

3.10 Operational Parameters

Three operational targets and one automated control mode govern the methodology’s behavior, and they are shared across substrates. The hallucination threshold is set to 0.55 for general-purpose domains and 0.58 for the legal and medical domains, reflecting the calibrated distribution of the embedding model against the production instruction format; the rationale is the subject of Section 4. The yield target is the proportion of generated outputs accepted after the full quality cascade, and the placeholder coverage target governs the proportion of in-scope PII tokens the masking layer recognizes and tokenizes, measured against the regulation-specific tier definitions documented in Section 5. The run mode is selected automatically by requested sample size — trial mode below 100 records, pilot mode from 100 to 999, and production mode at 1,000 or more — with the modes sharing every pipeline stage and differing only in logging verbosity, evaluation aggressiveness, and the strictness of completion checks.

The substrates differ in their generation batch size, and the difference is a direct consequence of their hardware profiles. SynthEval Studio operates at a smaller in-process batch sized to the RTX 4090’s memory-bandwidth-bound inference profile. SynthEval Cloud operates at a generation batch size of 128, empirically determined and hardware-confirmed to be the throughput optimum for Mistral-Nemo on the A10G under vLLM; the determination methodology and the telemetry confirming the engine is compute-bound at this batch size are documented in Section 6. A separate embedding batch size of 22 governs the local Arctic Embed forward pass on both substrates and is unrelated to the remote generation batch; it was deliberately left unchanged when the generation batch was tuned, because it addresses a different, local bottleneck.

3.11 Test Coverage

The methodology carries a pytest suite covering the configuration, generation, masking, quality, and evaluation modules. The most recent complete run passed in full at 187 of 187 tests, executed with per-test process isolation via `pytest --forked`. This is the suite’s clean baseline: every unit of behavior the tests assert is verified correct.

Process isolation is a required execution condition on the development host, and the reason is itself an architectural finding worth recording. Running the suite without process isolation produced non-deterministic crashes — segmentation faults and bus errors — originating inside third-party C extensions (the YAML C loader, the SciPy and scikit-learn numerical routines, and the regular-expression engine), not inside the pipeline’s own Python code. The cause was isolated to shared-fixture state across tests running in a single interpreter process, where repeated YAML parsing through the shared loader could corrupt interpreter state and crash a later, unrelated test. Running each test in its own forked process eliminates the shared state, and the suite passes 187 of 187 deterministically. The residual crashes seen without forking were confined to shared-fixture loading and were not defects in the code under test. This finding connects to a broader host-environment stability observation documented in Section 6: the pipeline logic is sound, while the development workstation exhibits occasional environmental faults, and the cloud substrate’s per-batch checkpointing

is what converts such faults from run-killers into recoverable interruptions.

3.12 Runtime Flow

Figure 3.3 traces the path of a single batch through the methodology from interface invocation to artifact materialization, showing where the two substrates diverge and where they converge. The flow is identical on both substrates except at the generation stage, where Studio invokes the in-process model and Cloud dispatches to the remote Modal vLLM service. Every stage downstream of generation — the embedding-driven hallucination gate, the PII masker and keyword tiers, the accept-or-reject routing, the evaluation pipeline, and the report builder — is the same on both, and runs locally in both.

Figure 3.3. Runtime data flow for a single batch through the SynthEval methodology. Configuration and profile resolution occur once at the start of a run. The generation stage is the only substrate-specific divergence: SynthEval Studio invokes the in-process model on the RTX 4090; SynthEval Cloud dispatches the batch to the remote Modal A10G vLLM service. The quality gate, masking, acceptance routing, and evaluation are identical and local on both substrates. SynthEval Cloud additionally checkpoints each batch’s accepted and rejected records to disk as the batch finalizes.

flowchart TD

```

A[CLI or HTTP API invocation] --> B[config_manager + profile_loader]
B --> C[orchestrator: run loop]
C --> D[batch_generator: assemble instructions + domain weighting]
D --> E{generation stage SUBSTRATE-SPECIFIC}
E -->|SynthEval Studio| F[in-process Mistral-Nemo 4-bit on RTX 4090]
E -->|SynthEval Cloud| G[remote Modal A10G vLLM Mistral-Nemo AWQ, batch 128]
F --> H[pipeline: raw outputs paired with instructions]
G --> H
H --> I[embedding_service: local Arctic Embed cosine similarity]
I --> J{hallucination gate threshold 0.55 / 0.58}
J -->|below threshold| K[rejected.jsonl reason: hallucination]
J -->|at or above threshold| L[pii_masker + keyword tiers]
L --> M{compliance cascade}
M -->|hard violation| N[rejected.jsonl reason: PII / bias / GDPR]
M -->|accepted| O[output.jsonl Cloud: checkpointed per batch]
N --> P[rejection_audit.json]
K --> P
O --> Q[evaluation/pipeline: diversity / quality / compliance yield / clustering]
Q --> R[report_builder]
R --> S[evaluation_report.html / .pdf / .json]
P --> T[run_manifest.json]
S --> T

```

The flow has three temporal phases. The initialization phase — configuration resolution, profile resolution, and generation-backend readiness — runs once at the start of a run and is amortized across every batch. The per-batch cascade — generation, embedding, hallucination gate, masking, and compliance routing — runs once per batch

and determines per-record throughput; in the cloud substrate it concludes with the checkpoint write. The finalization phase — evaluation, report rendering, and manifest writing — runs once at the end against the accepted corpus. Amortizing initialization across an entire run and deferring evaluation to the end is what enables the per-record economics documented in Section 2.3 and Section 6.

The remainder of this paper unpacks each subsystem named here. Section 4 documents the calibration of the hallucination gate. Section 5 documents the layered compliance filtration architecture and the independent audit. Section 6 documents the throughput, batch-size optimization, and economics of both substrates. Section 7 documents the evaluation pipeline and the contents of the report. Section 8 reports the results of the reference production runs on both substrates. Each subsequent section assumes the architectural vocabulary established here.

4 The Hallucination Gate

The first quality gate every generated record traverses is the hallucination filter. It is the component most directly responsible for the central commercial claim of the methodology — that a delivered record actually answers the instruction it was generated against — and it is the platform’s primary technical differentiator. This section documents what the gate measures, the dual-signal decision rule that determines acceptance, how the operating threshold was calibrated, and the production evidence across three runs that the gate discriminates as designed and transfers unchanged across both generation substrates.

4.1 Hallucination as Instruction-Output Drift

In a synthetic instruction-tuning corpus, a hallucinated record is one whose output does not respond to its instruction. The failure is not factual error in the abstract sense — the pipeline does not adjudicate truth claims about the world — but semantic drift between the instruction that was issued and the text that was generated in reply. A record instructing the model to draft a HIPAA breach-notification memo, answered with a generic privacy disclaimer that never addresses breach notification, is a hallucination in this operational sense: the output drifted off the instruction. For a buyer fine-tuning a model on the corpus, such records are worse than useless, because they teach the model to respond to a prompt with off-target content. Removing them is the difference between a dataset that improves a model and one that degrades it.

Detecting this drift at scale requires a measure of semantic correspondence between instruction and output that can be computed automatically for every record. The methodology uses embedding cosine similarity as that measure, computed by the local embedding service against the calibrated, domain-specific threshold documented below.

4.2 The Dual-Signal Decision Rule

The gate is not a simple cosine floor. It is a two-signal decision in which a record is rejected as hallucinated only when both an embedding-similarity check and a keyword-overlap check fail simultaneously. This design is the single most important property of the gate, and it is what allows the filter to achieve high precision against off-topic drift without sacrificing recall on terse, on-topic, domain-specific outputs.

The first signal is embedding cosine similarity. The embedding service computes a CLS-pooled 1024-dimensional vector for the instruction and for the output using the Snowflake Arctic Embed L v2.0 model, applying the model-recommended query: prefix to the instruction, and takes the cosine similarity between the two vectors. The similarity is computed on the raw generated output, before the masking layer substitutes any personal-data spans, so the score reflects the model’s actual response rather than its post-masking form. This raw-output cosine is the value persisted to the published record as its quality signal. The first signal fails when this similarity falls below the record’s domain threshold.

The second signal is a lightweight keyword-overlap check that serves as a semantic safety net. It asks whether the output contains at least one substantive token — a word longer than three characters — drawn from the leading tokens of the instruction. The check is deliberately simple and conservative: it confirms that the output is at minimum lexically anchored to the instruction’s subject, even where the embedding similarity is low.

The decision rule combines the two as a logical AND:

A record is rejected as hallucinated only if the cosine check fails **and** the keyword-overlap check also fails.

The consequence is that a low cosine score alone does not reject a record. If the output is lexically anchored to the instruction — if the keyword-overlap check passes — the record is accepted even when its cosine similarity sits below the domain threshold, on the reasoning that an output sharing the instruction’s substantive vocabulary is very likely on-topic despite an embedding score the model under-assigned. The cosine threshold is therefore a necessary but not sufficient condition for rejection. This is what makes the gate precision-oriented against genuine drift while preserving recall on legitimate outputs that the embedding model happens to score conservatively, a pattern most common in domains whose dense, specialized vocabulary the general-purpose embedder represents less confidently.

4.3 Calibrating the Threshold

The operating threshold was not inherited from a literature default or a prototype value; it was calibrated against the embedding model’s behavior on this pipeline’s own instruction-output distribution. An early development threshold of 0.18 — chosen before the pipeline’s true similarity distribution was understood — proved far too permissive: at that level the cosine signal admitted nearly everything and did no discriminating work, because the great majority of genuine instruction-output pairs score well above it. Examining the empirical distribution of similarities the

production pipeline actually produces showed that legitimate, on-topic outputs cluster in a band approaching and exceeding 0.7, while genuinely off-topic outputs fall substantially lower. The threshold was recalibrated upward to sit at the boundary between those populations, where it separates drift from on-topic generation rather than rubber-stamping both.

The calibrated operating thresholds are 0.55 for the general-purpose domains and 0.58 for the legal and medical domains. The higher floor for legal and medical reflects a property of the embedding model rather than a stricter quality standard: those domains carry the densest, most specialized vocabulary, and the general-purpose embedder represents their outputs with systematically lower instruction-output similarity even when the output is correctly on-topic. Raising the floor for those two domains compensates for that representational effect. The per-domain thresholds are declared in configuration, not hard-coded, so the boundary for any domain can be retuned without code changes as additional distributional evidence accumulates.

It is precisely at this higher legal-medical floor that the dual-signal rule does its most visible work, as the production evidence below shows: the records the keyword safety net rescues from rejection are overwhelmingly legal and medical outputs whose specialized vocabulary depressed their cosine score below 0.58 even though they were on-topic.

4.4 Production Validation

The gate’s behavior was validated across three production runs: one local run on SynthEval Studio (run_20260521_230610, 1,000 accepted records) and two cloud runs on SynthEval Cloud (run_20260601_015003 and run_20260602_010756, 1,110 and 1,116 accepted records respectively). The local run serves as the calibration and discrimination anchor; the two cloud runs serve as production-scale validation that the calibrated threshold transfers across the serving substrate. The figures in this section are taken directly from the per-record cosine values persisted in each run’s output and rejection artifacts.

Across all three runs the gate exhibits the same behavior. The accepted records cluster tightly with a mean instruction-output cosine of approximately 0.73 — well above the 0.55 general floor — while the threshold and the dual-signal rule together reject the small fraction of records exhibiting genuine drift. Table 4.1 summarizes the gate’s behavior across the three runs.

Table 4.1. *Hallucination-gate behavior across three production runs. The threshold and accepted-cosine statistics are identical in character across substrates; the accepted-versus-rejected cosine separation is cleanest in the local run because, as discussed below, a larger share of the cloud rejections are non-hallucination rejections that dilute the rejected-cosine mean.*

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
Threshold (general / legal-med)	0.55 / 0.58	0.55 / 0.58	0.55 / 0.58
Hallucination rejections	8 (0.79%)	8 (0.69%)	10 (0.87%)
Accepted cosine mean	0.734	0.732	0.732

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
Accepted cosine median	0.741	0.740	0.738
Rejected cosine mean	0.643	0.707	0.681
Accepted – rejected separation	+0.091	+0.024	+0.051
Overall yield	98.3%	96.4%	96.9%

Figure 4.1 shows the full instruction-output cosine distribution for the local anchor run. The accepted records form a tight distribution centered near 0.73. The seventeen rejected records, by contrast, do not cluster at any single cosine value, because rejection is driven by several gates, not the hallucination gate alone: the lowest-cosine rejections are the genuine hallucinations the cosine signal catches, while rejected records with mid- and high-range cosine scores were removed by the privacy, bias, GDPR, or fidelity gates and carry no hallucination penalty. The hallucination gate’s own discrimination is seen in the separation of means — accepted 0.734 versus rejected 0.643 — and most cleanly in the low-cosine tail, where records that fail both the cosine and keyword signals are removed.

Figure 4.1. *Instruction-output cosine similarity distribution for the local anchor run (run_20260521_230610, 1,000 accepted, 17 rejected). Accepted records (blue) center on a mean of 0.734; rejected records (red) center on 0.643 and concentrate below the 0.55 general-domain threshold (dashed line; the 0.58 legal-medical threshold is shown dotted). The separation between the populations is the gate’s discrimination made visible.*

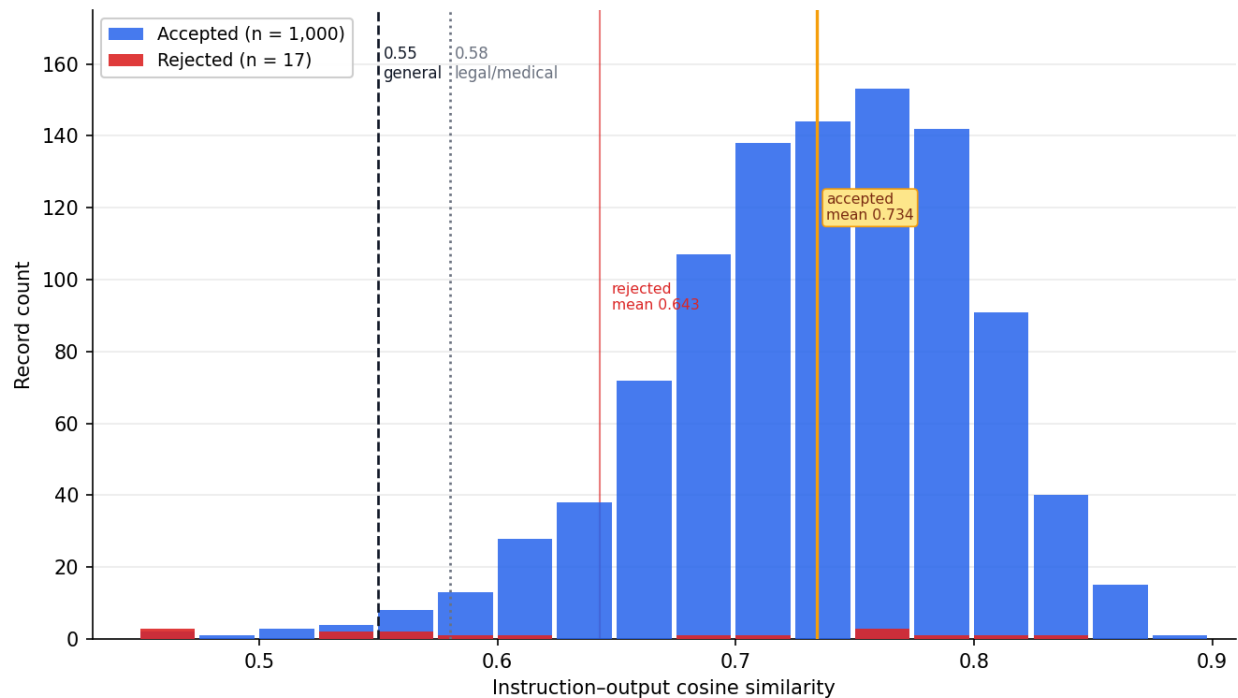


Figure 3: Figure 4.1 — Cosine similarity distribution, local anchor run

The accepted-versus-rejected cosine separation is largest in the local run, at +0.091,

and smaller in the two cloud runs, at +0.024 and +0.051. This difference is not a degradation of the gate on the cloud substrate; it is an artifact of rejection composition. The hallucination gate is only one of several rejection paths — records are also rejected for privacy, bias, GDPR, URL, and placeholder-fidelity violations — and a record rejected for a non-hallucination reason can carry a perfectly high instruction-output cosine. In the two cloud runs a larger share of total rejections were non-hallucination rejections, which raises the mean cosine of the rejected set and compresses the accepted-versus-rejected gap. The cleanest measure of the hallucination gate’s discriminating power is therefore the local run, where the rejected set is more concentrated on genuine drift; the cloud runs confirm that the accepted population is stable across substrate, with accepted-cosine means within three thousandths of one another across all three runs.

The dual-signal rule’s effect is directly observable in the accepted sets. In every run, a small fraction of accepted records carry a cosine similarity below their domain threshold — 1.8% of accepted records in the local run, 2.4% in the first cloud run, and 1.8% in the second — and were admitted because the keyword-overlap safety net confirmed them on-topic. These rescued records are overwhelmingly concentrated in the legal and medical domains: in the local run, all eighteen sub-threshold acceptances were legal or medical; in the two cloud runs, the same pattern held, with the large majority falling in those two domains. This is the dual-signal design working exactly as intended. The records the keyword net rescues are precisely the terse, vocabulary-dense legal and medical outputs that a cosine floor alone would wrongly discard, and the gate recovers them without weakening its rejection of genuine drift, which continues to fail both signals and is removed.

4.5 Substrate Independence of the Gate

A property worth stating explicitly is that the hallucination gate is independent of the generation substrate. The embedding model that computes the cosine signal runs locally on the workstation in both deployments — even in SynthEval Cloud, where token generation is offloaded to the remote Modal A10G service. The gate therefore evaluates Studio-generated and Cloud-generated records on identical local hardware, with the identical embedding model, the identical thresholds, and the identical dual-signal rule. The consistency visible in Table 4.1 — accepted-cosine means within three thousandths across all three runs, identical thresholds, and hallucination-rejection rates between 0.69% and 0.87% — is the empirical confirmation that the quality signal does not depend on where tokens are produced. This is the hallucination-gate instance of the paper’s broader thesis: the methodology is portable across substrates because the quality layer is substrate-independent by construction.

The throughput and quality consequences of the quantization format chosen for each substrate — the comparison of GGUF, BitsandBytes, and AWQ-vLLM builds on both generation speed and output quality — are documented in Section 6, where the full quantization-format selection analysis is presented alongside the batch-size optimization and economics. The present section establishes only that the calibrated gate discriminates as designed and transfers unchanged across substrates; Section 7 documents how the gate’s per-record outputs feed the evaluation pipeline and the attested report.

5 The Compliance Architecture

The second quality layer every record traverses is compliance filtration. Where the hallucination gate of Section 4 asks whether an output answers its instruction, the compliance layer asks whether the output is safe to deliver: whether it carries raw personal data, severe bias, or regulated content that a buyer in a regulated industry cannot accept. This section documents the layered masking architecture, the severity-tiered keyword filters, the placeholder-fidelity checks, the record-level acceptance model that combines them, and the independent audit that verifies the result. It closes with the compliance metrics computed directly from three production runs, which show the architecture functioning as a dataset-cleaning mechanism rather than a constraint on the generator.

5.1 Compliance as a Gate, Not a Disclaimer

The defining property of this layer is that compliance is enforced at generation time, against named regulatory frameworks, and verified on the delivered output — it is not a license disclaimer appended to a dataset card. The masker operates against an active regulatory profile (GDPR, HIPAA, or CCPA, selected in configuration) and applies framework-appropriate detection to every generated record before that record is eligible for delivery. A record is admitted to the published dataset only after its detectable personal data has been replaced with typed placeholders and after it has cleared the keyword and fidelity checks. Records that cannot be made compliant are not delivered; they are written to the run’s rejection log. The delivered dataset is therefore clean by construction, and the cleanliness is a measured property of the output rather than an assertion about intent.

This design rests on a deliberate separation of concerns. The generator is allowed to generate naturally, without being clamped into a narrow “safe” vocabulary that would degrade diversity and realism. The compliance layer then removes or masks what does not pass. The dataset is made clean by subtraction at the gate, not by coercion of the model — a distinction that matters for both quality and credibility, and one the production results in §5.7 make concrete.

5.2 Layered PII Masking

The masking entry point is `pii_masker.py`, which applies domain-aware, typed-placeholder substitution to each output. Detected personal data is replaced with a labeled placeholder — `<EMAIL>`, `<IP_ADDRESS>`, `<NATIONAL_ID>`, `<CPT_CODE>`, and so on — so that the structure and realism of the record are preserved while the sensitive value is removed. Each placeholder type also carries an implicit regulatory mapping: an email address is personal data under GDPR, CCPA, and HIPAA simultaneously, while a CPT procedure code is primarily a HIPAA concern, and the placeholder vocabulary encodes those associations.

Two architectural properties make the masking robust rather than cosmetic. The first is an always-mask floor: a small set of high-sensitivity identifier types — Social Security numbers, phone numbers, email addresses, credit-card numbers, and IP addresses — are masked in every domain regardless of that domain’s configured

allowed-placeholder list. The per-domain allowed and forbidden sets govern only the softer, domain-contextual placeholders; the always-mask floor is non-negotiable and domain-independent, so a high-sensitivity value cannot pass merely because a domain’s configuration did not anticipate it.

The second property is span protection for mixed records. A record frequently contains some correctly placed placeholders alongside a raw value that escaped substitution — a masked email beside an unmasked IP address in a table cell, for example. An earlier version of the detection logic treated the presence of any placeholder as evidence the record was clean, which blinded it to such mixed records. The current architecture blanks the existing placeholder spans to equal-length gaps and then scans the remaining text for raw personal data, so a partially masked record is still checked for leaks in its unmasked remainder. This span-protection logic extends to truncated identifiers — an IP address cut off at a token boundary, such as a three-octet fragment ending a table field — which are anchored and detected rather than missed. These two properties are what allow the masker to handle the realistic, messy output a generator actually produces.

5.3 Severity-Tiered Keyword Filters

Beyond personal-data masking, the compliance layer applies two severity-tiered keyword filters that operate on the content of the output rather than on identifier patterns.

The bias filter maintains two tiers. The severe tier contains terms whose presence in a delivered record is categorically unacceptable, and any severe-tier match rejects the record outright in every domain. The moderate tier contains terms that are sensitive but legitimately appear in professional discourse — terms such as *racial profiling*, *disability*, or *gender dysphoria* that a legal brief, an educational policy, or a transport-oversight document may need to use in good faith. Moderate-tier matches reject the record in general domains but are exempted in the education, legal, and transport domains, where such terminology is the legitimate subject matter rather than a bias signal. This domain-aware exemption prevents the filter from rejecting on-topic professional content while still catching gratuitous use elsewhere.

The GDPR keyword filter is likewise tiered, but with an asymmetry specific to its purpose. The severe tier — terms denoting regulatory penalty exposure — triggers rejection. The moderate tier — the standard vocabulary of data-protection practice, such as *data subject*, *lawful basis*, *right to erasure*, and *data protection impact assessment* — is informational only and never causes rejection. This is a deliberate and important choice: in a dataset whose explicit purpose includes GDPR-relevant content, the appearance of GDPR terminology is the subject matter, not a violation. Flagging it for rejection would gut the GDPR domain. The moderate tier is therefore recorded for reporting — it surfaces as the GDPR-surface metric in §5.7 — without gating delivery.

5.4 Placeholder Fidelity

The third compliance check is placeholder fidelity, computed by the masker's `check_fidelity` routine. It verifies two things about each masked record. First, that the record contains no forbidden placeholder types for its domain: a medical record carrying a `<LICENSE_PLATE>` or an ecommerce record carrying an `<ICD10_CODE>` indicates the generator drifted out of domain, and the forbidden-placeholder set for each domain is declared in configuration. The presence of any forbidden placeholder type fails fidelity. Second, it measures placeholder density — the count of placeholders in the record — against the domain's configured minimum. For a defined set of required formats whose templates are built to carry placeholder tokens, a density below the domain minimum indicates the record under-populated its expected compliance surface, and such a record is routed to regeneration rather than accepted as-is. Formats whose nature is operational prose, not placeholder-bearing, are exempt from the density floor and accepted at zero placeholders. The fidelity check is therefore both a domain-correctness signal and the trigger for the density-driven regeneration path described next.

5.5 The Three-Category Acceptance Model

The masking, keyword, and fidelity checks combine into a single record-level acceptance model with three outcomes, applied uniformly to first-pass records and to re-generated records alike.

The first outcome is hard rejection. A record carrying a hard violation — raw PII surviving the masker, a severe bias match, a moderate bias match outside the exempt domains, a severe GDPR match, a risky-URL pattern, or a forbidden placeholder for its domain — is rejected outright and written to the run's `rejected.jsonl` with its reason. It is never delivered. This is the path that does the cleaning: the non-compliant record is removed from the dataset, not repaired in place.

The second outcome is masking in place. A record whose only personal-data exposure is detectable and substitutable has those spans replaced with typed placeholders and is admitted. The record is delivered in masked form, with the raw value gone and the structural realism preserved.

The third outcome is regeneration. A record with no hard violation but a placeholder density below the floor for a required format is not discarded; it is collected and regenerated in a batched retry round, then re-gated against the identical checks. A retried record that clears the density floor and carries no hard violation is admitted; one that still fails is rejected. The regeneration path is what allows the pipeline to recover recoverable records — those that were merely sparse rather than unsafe — without weakening any safety gate, because the retried record passes through the same acceptance model as a first-pass record.

The consequence of this model is the framing that the production results below make measurable: the metrics reported on the delivered dataset are the residual that survived a working filter, and the records that did not survive are accounted for in the rejection log. A clean delivered dataset is the product of removal at the gate, not of constraining the generator upstream.

5.6 Independent Leak Auditing

The most important property of the compliance layer is that its verification does not trust its own masker. The leak-detection metric in `compliance.py` uses its own independent set of raw-PII patterns — not the masker’s patterns — and scans the placeholder-stripped remainder of every delivered record for personal data that survived masking. This independence is deliberate and is the resolution of a hard-won lesson: when a leak detector reuses the masker’s own patterns, it can report a perfectly clean result while a raw value is present, because any value the masker failed to catch is, by definition, a value its patterns do not match. A privacy metric that can under-report in exactly the cases that matter is worse than the leak it conceals. Aligning the detector to be at least as strong as the masker, and validating with patterns deliberately broadened to over-report rather than under-report, is what makes the resulting leak rate trustworthy.

This independent verification exists in two forms. Within the pipeline, the compliance metric scans every delivered record and reports the residual PII leak rate as part of the evaluation report. Separately, a standalone audit script, `audit_pii.py`, performs an independent raw-PII leak audit using the same deliberately broad patterns, providing a verification path that does not run inside the generation pipeline at all. A third script, `mask_audit.py`, supports reproducibility for external review: it consumes a curated test corpus, applies the PII patterns independently, and renders a standalone HTML report showing each record’s before-and-after text, which patterns fired, and which regulatory framework each placeholder satisfies — a demonstration artifact that lets a reviewer inspect the masking behavior directly rather than taking the leak rate on faith.

5.7 Production Compliance Results

The compliance metrics below were computed directly from the delivered records of three production runs using the pipeline’s own independent compliance scanner — one local run on SynthEval Studio (`run_20260521_230610`) and the two cloud runs on SynthEval Cloud (`run_20260601_015003` and `run_20260602_010756`). The local run is shown as a pre-hardening baseline; the two cloud runs were generated after the masking architecture’s span-protection and truncated-identifier detection were implemented. All rates are measured on the accepted, delivered dataset — they are the residual that survived into the clean product, not a count of what was rejected.

Table 5.1. *Compliance metrics across three production runs, computed by the independent compliance scanner on the delivered records. The local run is the pre-hardening baseline; the two cloud runs are post-hardening. The PII leak rate is the residual raw personal data detected in the delivered set by the independent scanner.*

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
Residual PII leak rate	3.8%	0.72%	0.0%
Echo contamination rate	0.0%	0.0%	0.0%
Placeholder coverage	87.3%	51.5%	52.5%
Placeholder density (mean)	4.78	2.91	2.91

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
GDPR-surface rate	39%	45%	48%
Bias-flag rate	1.4%	1.8%	2.06%

The headline result is the residual PII leak rate, shown in Figure 5.1. Under the pre-hardening masker, the local run delivered a 3.8% residual leak rate — a non-trivial fraction of records carrying raw personal data the masker did not catch. After the span-protection and truncated-identifier hardening, the first cloud run fell to 0.72%, and the second cloud run reached 0.0%: the independent scanner found no raw personal data surviving into the delivered dataset. The reduction is the masking architecture working as intended — records whose personal data could be masked were masked, and records carrying raw values the masker could not resolve were rejected to the run’s rejection log rather than delivered. The zero in the second cloud run is meaningful precisely because it was measured by a scanner using deliberately broader patterns than the masker, designed to over-report; a zero from that scanner is evidence of a clean delivered set, not of a metric blind to its own gaps.

Figure 5.1. *Residual PII leak rate in the delivered dataset across the three production runs, measured by the independent compliance scanner. The reduction from 3.8% (pre-hardening) to 0.72% and then 0.0% (post-hardening) reflects the span-protection and truncated-identifier detection added to the masking layer.*

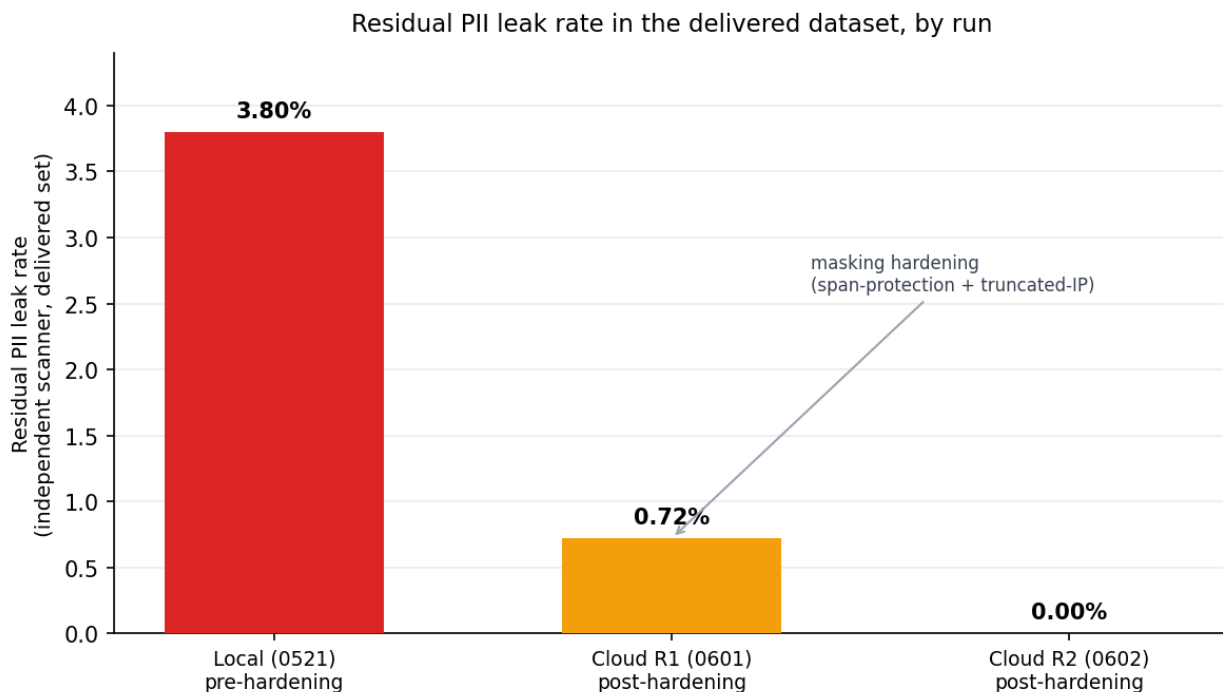


Figure 4: Figure 5.1 — Residual PII leak reduction across runs

The residual leak rate of 0.72% in the first cloud run, before reaching zero in the second, is itself instructive about the scanner’s design. The handful of records it

flagged were dominated by values the broad scanner is built to over-report: private and reserved-range IP addresses such as 192.168.x.x, and the canonical placeholder Social Security number 123-45-6789, alongside a small number of genuine email escapes. The scanner deliberately does not range-validate IP octets or whitelist documentation domains, because the cost of a false positive — a record needlessly flagged for review — is trivial next to the cost of a false negative, a real leak reported clean. This bias toward over-reporting is the correct posture for a privacy audit, and it is why the eventual zero is credible.

The placeholder coverage and density figures require interpretation, because they fall from the local run to the cloud runs and a naive reading would mistake that for a regression. Placeholder coverage is the fraction of delivered records carrying at least one placeholder, and density is the mean placeholder count per record. The local run’s 87.3% coverage and 4.78 density reflect an over-dense generation regime that the diversity analysis in Section 7 identifies as a driver of lexical repetition. The cloud runs land at a mean density of 2.91, almost exactly the configured target of 2.5 placeholders per record, with coverage near 52%. The change is the density-control configuration pulling generation toward its intended target, not a loss of masking capability — every record that contains personal data is still masked, while records whose natural content carries little or no personal data are no longer pushed to manufacture placeholders they do not need. The downstream domain-rebalancing and post-processing workflow operates on the delivered records and is documented in Section 7.

The remaining metrics are stable and expected. Echo contamination — a delivered output that merely restates its instruction — is zero across all three runs. The GDPR-surface rate rises across the runs, from 39% to 48%, reflecting a larger share of records surfacing GDPR terminology; as established in §5.3, this is the legitimate subject matter of the GDPR domain and is reported, not gated. The bias-flag rate sits between 1.4% and 2.06%, and this figure is reported on the delivered set in the spirit of §5.1: it is a small, measured, residual signal disclosed honestly rather than suppressed, and it falls below the rejection threshold by the domain-aware tiering of §5.3. Reporting it transparently, rather than claiming an unqualified zero, is the difference between a defensible compliance claim and an indefensible one — a distinction Section 10 addresses directly as part of the methodology’s residual-risk posture.

6 Throughput, Economics, and Quantization Format Selection

This section establishes the operational case for SynthEval Cloud: that a correctly-engineered cloud deployment beats local generation on throughput and cost *without* sacrificing the quality bar established during local development. The case rests on two engineering decisions made on measured evidence rather than convenience — the choice of quantization format, and the choice of generation batch size — and on a triangulated cost model derived from production runs. The central claim is specific and testable: at quality parity (MATTR $\approx$ 76%), the cloud pipeline generates roughly 13 $\times$ faster than local at the lowest per-record cost of any format tested.

6.1 The Problem: Local Throughput Fails Time-Bound SLAs

The local SynthEval Studio pipeline, running a 4-bit model on a single RTX 4090, produces high-quality datasets — a measured 76.4% MATTR — at approximately **251 prompts per hour**. For small datasets this is entirely adequate and, on a per-record basis, extremely cheap (~\$0.13 per 1,000 records in local compute). But throughput scales linearly with wall-clock time, and large client deliverables expose the limit: a 20,000-record dataset requires on the order of **five days** of continuous local generation. That duration fails time-bound client SLAs outright. The cloud pipeline exists to break that wall — but only if it can do so while holding the quality bar local development established, because a faster pipeline that ships measurably worse data solves nothing.

6.2 Model Selection

The generation model was chosen before the quantization format, and like the format it was selected on measured evaluation performance rather than on reputation or parameter count. The principal alternative evaluated was Qwen-2.5-32B-Instruct, a model nearly three times the parameter count of Mistral-Nemo-Instruct-2407, tested in three separate configurations to isolate the effect of quantization from the effect of the model itself: full-precision F16, BitsandBytes NF4 quantized at load time, and a pre-quantized BitsandBytes NF4 build. The NF4 configuration was constructed deliberately as a direct comparison point, using the identical quantization scheme applied to Mistral-Nemo, so that the two models could be compared on equal quantization footing rather than across confounded precisions. Because the 32B model exceeds the memory of the A10G in these configurations, its evaluation ran on a larger A100-40GB GPU, while Mistral-Nemo ran on the A10G — meaning the larger competitor was given the more capable hardware and still did not prevail.

Across the pipeline’s own evaluation framework — the same diversity, quality, and yield metrics documented in Section 7, applied identically to every candidate — Mistral-Nemo-Instruct-2407 produced the best results across the board, and did so most decisively in local generation. This is a consequential finding rather than a convenient one: a 12-billion-parameter model outperforming a 32-billion-parameter alternative on the evaluation metrics that matter for this task is what makes the entire economic geometry of the methodology possible. The smaller model fits the cheaper A10G, sustains higher throughput under continuous batching, and carries a lower per-record cost, and it does so without surrendering quality to the larger model. Had the 32B model won on quality, the cost and throughput advantages documented in the remainder of this section would have been unavailable, because serving a 32B model requires more expensive hardware at lower throughput. The model selection is therefore not a preliminary detail but a load-bearing decision: Mistral-Nemo was retained because it measured best, and its measuring best is precisely what permits the cost-efficient cloud substrate that follows.

6.3 Quantization Format Selection

The quantization format sets the ceiling on both quality and throughput before any other tuning. Three formats of the same model (Mistral-Nemo-Instruct-2407)

were measured end-to-end through the identical evaluation framework on the same A10G GPU: **GGUF Q4_K_M** via llama-cpp, **BitsandBytes 4-bit NF4** via HuggingFace Transformers, and **AWQ** (compressed-tensors, asymmetric W4A16) via vLLM. Quality and throughput were weighed together, because each format trades them differently.

6.3.1 Side-by-Side Results

Metric	GGUF Q4_K_M	BitsandBytes NF4	AWQ-vLLM
Serving backend	llama-cpp	HF Transformers	vLLM
MATTR (quality)	68.70%	76.29%	76.8-76.9%
Δ vs local (76.4%)	-7.7 pts	-0.11 pts	$\approx$ match
Repetition rate	1.0%	3.0%	2.4-3.1%
Avg cosine similarity	0.733	0.733	0.732
Throughput	486 / hr	168 / hr	$\sim$ 3,290 / hr
Δ vs local (251/hr)	1.94 $\times$	0.67 $\times$	$\sim$ 13 $\times$
Cost / 1,000 records	$\sim$ \$2.30	$\sim$ \$6.54	$\sim$ \$0.43-\$0.48
PII leak rate	0.0%	0.0%	0.0%
Decision	Rejected	Local only	Selected (cloud)

6.3.2 GGUF — Fast but Quality-Capped

GGUF was the readily-available format and the fastest to stand up, running at 1.94 $\times$ local throughput. Four tuning iterations established that its quality could not reach the bar. The naive first configuration produced **62.68% MATTR** at 4% repetition. Adding a repetition penalty recovered six points to **68.70% MATTR** at 1% repetition — its best achievable result. Pushing further (a structured system prompt at a larger token budget) regressed every diversity metric and drove repetition to 13%. The conclusion is structural: **68.7% MATTR is the hard ceiling of Q4_K_M quantization**, about 7.7 points below local, a deficit caused by the lossiness of the k-quant format itself and not removable by tuning. GGUF buys speed at a quality cost that cannot be tuned away.

6.3.3 BitsandBytes — Parity but Slow and Costly

Deploying the exact 4-bit NF4 model the local pipeline uses recovered the quality fully — **76.29% MATTR, within 0.11 points of local**, and 7.6 points above the GGUF ceiling. But its throughput collapsed to **168 prompts per hour** — slower than the local 4090 — at **$\sim$ \$6.54 per 1,000 records**, the most expensive of the three. BitsandBytes proved cloud quality parity was attainable, but at an operational profile that defeats the purpose of moving off local hardware.

6.3.4 AWQ-vLLM — Breaking the Tradeoff

AWQ served through vLLM resolved the dilemma the first two formats framed. It held quality at **76.8-76.9% MATTR** (parity with both BitsandBytes and local) while running at **$\sim$ 3,290 prompts per hour — roughly 13 $\times$ local — at $\sim$ \$0.43-\$0.48 per**

1,000 records, the lowest cost of any format. vLLM’s paged-attention batched serving and AWQ’s activation-aware quantization together deliver both axes at once. (The asymmetric W4A16 model pins vLLM 0.8.5.post1, whose scheme matcher accepts asymmetric weights where earlier versions did not.) AWQ-vLLM is the production cloud format; BitsandBytes NF4 remains the local Studio format.

6.4 Batch-Size Optimization (AWQ-vLLM)

With the format chosen, generation throughput is governed by batch size — the number of prompts sent to the vLLM engine per inference call. A controlled sweep, using real domain-varied prompts at the production token budget against the deployed engine, found a single-peaked curve: throughput rises to a maximum at **batch size 128** and then declines as larger batches add coordination overhead without usable concurrency.

Batch size	Behavior	Throughput
128	Optimum	Peak (~1,150–1,260 tok/s in production)
192–256	Declining	~773–795 tok/s (benchmark)
384–512	Declining further	~752–765 tok/s (benchmark)
768	Beyond practical limit	Exceeds 600 s call timeout

Live GPU telemetry during a production run confirms why 128 is optimal rather than merely fastest in the sweep (Figure 6.1). At batch 128 the A10G runs at **~95–100% utilization and ~100% power utilization while holding ~20 GB of 24 GB**, with GPU temperature plateauing near its steady-state ceiling for the duration of generation and dropping sharply the moment the run completes. The card is compute-bound — fully worked but not at the memory edge — so larger batches cannot run faster and instead push toward the memory ceiling and scheduler overhead, which is why throughput declines past 128. The synchronized fall in utilization, power, and temperature at the end of the run, against a flat ~20 GB memory hold, is the visual signature of a workload bounded by compute rather than memory.

Figure 6.1. *Modal A10G GPU telemetry during a production AWQ-vLLM run at batch size 128. GPU utilization holds in the 95–100% band and power utilization sits at ~100% throughout generation, while GPU memory stays flat at ~20 GB of 24 GB and temperature plateaus near its steady-state ceiling — the signature of a compute-bound workload. All four signals fall together when the run completes (right edge).*

6.5 Vertical Scaling: Larger GPU Tiers

The batch-size optimum and the throughput it yields are specific to the A10G. Because the A10G runs compute-bound at batch 128 — fully worked, with memory held well below capacity, as the telemetry in §6.3 shows — the workload scales with the compute and memory-bandwidth headroom of a larger card. Moving the same methodology to a higher GPU tier therefore raises throughput without any change to the pipeline above the generation call.



Figure 5: Figure 6.1 — A10G telemetry at batch 128, production run

Projecting that scaling conservatively — using the lower of the compute-throughput and memory-bandwidth ratios from published vendor specifications, the defensible floor rather than the optimistic ceiling, and assuming the generation batch size is re-tuned to each card’s optimum with the same `benchmark_batch_size.py` sweep used for the A10G — an NVIDIA A100 80GB projects to approximately 8,200 prompts per hour and an NVIDIA H100 80GB to approximately 18,400, against the A10G’s measured $\sim 3,290$. The cost behavior is the decisive part: Modal’s hourly rate rises with GPU tier, but throughput rises faster, so the all-in cost per thousand records *falls* as the card gets larger — approximately \$0.45 on the A10G, \$0.41 on the A100, and \$0.29 on the H100. A one-million-record deliverable, fully gated, masked, evaluated, and audited, projects to roughly \$450 and 12.6 days on the A10G, \$412 and 5.1 days on the A100, or \$290 and 2.3 days on the H100. The full cross-tier table, including frontier-API reference figures, is presented as Table 2.3 in §2.3.1.

Moving to a larger card is a configuration change, not a re-engineering effort, precisely because the methodology layer is hardware-independent. In practice it requires only changing the GPU type in the Modal service declaration, re-running the batch-size sweep to find the new optimum, widening the context-length and key-value-cache settings to use the larger memory, and re-measuring per-record cost against the new hourly rate; an optional FP8 or higher-precision model variant becomes available on these cards but is not required. The hallucination gate, the masking architecture, the evaluation pipeline, and the audit are untouched. These figures are projections grounded in the measured compute-bound behavior of the A10G and published GPU pricing, not measured runs on the larger cards, and are presented as such.

6.6 Horizontal Scaling: Concurrent Containers

Batch size optimizes a single generation container. Throughput scales on a second, independent axis by running multiple containers concurrently. Because the cloud substrate dispatches generation to a serverless GPU service, the deployed vLLM engine can be replicated across many containers, each processing batches in parallel, by raising the service’s maximum-container limit. Aggregate throughput then rises in proportion to the container count: the measured single-container rate of $\sim 3,290$ prompts per hour becomes approximately $N \times 3,290$ across N concurrent containers, and matching any target delivery rate — including the effective throughput of a frontier generation API — becomes a question of provisioned concurrency rather than a different pipeline. Reaching an effective rate of R records per hour requires on the order of $R / 3,290$ containers.

Per-record cost stays approximately flat under this scaling, because cost and throughput rise together: N containers cost N times the hourly rate but produce N times the records, so the $\sim \$0.45$ per 1,000-record figure is preserved while wall-clock delivery time falls by a factor of N . Horizontal scaling therefore buys delivery speed without buying it at a worse unit economics — the distinguishing property that separates it from simply renting a faster API.

Two honest bounds apply. First, this is a projection from measured single-container behavior, not a measured multi-container run; the linear-scaling assumption holds only until a shared resource saturates. The binding constraint in practice is the local

evaluation stage: the Arctic-embedding hallucination gate and the evaluation pipeline run on the local workstation, and at high generation concurrency that local stage becomes the bottleneck unless it is itself parallelized or batched to keep pace. Second, provider-side container limits and rate ceilings cap the achievable concurrency in any given account tier. With those two caveats stated, the architectural point stands: the cloud substrate’s delivery speed is not fixed at the single-container rate, and approaching frontier-equivalent aggregate throughput is a matter of concurrency and evaluation-stage scaling, not a change to the methodology. This horizontal axis complements the vertical GPU-tier scaling projected in Section 2.3.1 (Table 2.3): vertical scaling raises single-stream speed with a larger GPU, horizontal scaling multiplies streams with more containers, and the two compose.

6.7 Cost Model (Triangulated)

The all-in cost of a 1,000-record AWQ-vLLM run was measured three independent ways, agreeing within the container idle-tail margin:

Method	Result / 1,000	Captures
Credit-balance delta	~\$0.43-\$0.48	True billed cost incl. cold start + scaledown idle
Generation time × full rate	~\$0.38	Pure compute (GPU + ~1 core + ~2 GB)
Generation + cold start × rate	~\$0.39	Compute plus one-time container start

The GPU dominates the rate; CPU and host memory add only ~5.7% over the GPU-only A10G rate. The gap between the ~\$0.45 measured credit delta and the ~\$0.38 computed compute cost is the scaledown idle tail. Per-record marginal cost is ~\$0.00034, so a **20,000-record deliverable — the workload that takes ~5 days locally — projects to roughly \$7-\$10 of compute and completes in hours, not days.**

6.8 The Operational Case, Stated Plainly

Putting the three formats on both axes against local makes the conclusion unambiguous:

- **GGUF:** 1.94× local speed, but −7.7 MATTR — fails the quality bar.
- **BitsandBytes:** quality parity, but slower than local at the highest cost — fails the throughput purpose.
- **AWQ-vLLM:** quality parity at ~13× local throughput and the lowest cost — meets both.

The cloud pipeline therefore delivers what the local pipeline structurally cannot: large datasets inside time-bound SLAs, at a verified quality bar, at lower per-record cost than either the slow cloud alternative or the API providers it competes with — and, because throughput scales horizontally across concurrent containers (§6.4) as well as vertically across GPU tiers, that delivery speed is bounded by provisioned concurrency rather than by the pipeline itself. And the reason the decision is sound rather

than lucky is the measurement framework itself: a naive deployment reaching for GGUF would have shipped a 62.7% MATTR dataset and not known it. Because local development had already established 76%+ as the standard and the evaluation pipeline could quantify the shortfall, the format choice was driven by data. The measurement infrastructure is the product — it is what converts “it runs in the cloud” into “it runs in the cloud at a proven quality bar, an order of magnitude faster than local, for cents per thousand records.”

All figures measured: GGUF runs R1-R4 (run_20260523-20260524), BnB4bit R5 (run_20260525_011036), AWQ-vLLM production runs (run_20260601_015003, run_20260602_010756), the batch-size benchmark sweep, A10G dashboard telemetry, and the triangulated credit-delta cost analysis.

7 The Evaluation Pipeline and the Attested Report

Every dataset SynthEval delivers is accompanied by a machine-readable evaluation report generated by the same pipeline that produced the data. This is the artifact that converts “the records passed the gates” into something a buyer can audit: a structured, reproducible account of what was generated, what was rejected and why, how diverse and well-formed the accepted records are, and whether the run’s own internal counts reconcile. This section documents the evaluation pipeline’s architecture, the five metric families it computes, the reconciliation step that guards against silent data loss, a worked example drawn from a production run, and the diversity-analysis episode in which the pipeline caught a real generation defect and drove the methodology’s metric design.

7.1 Evaluation as a Deliverable, Not a Diagnostic

The evaluation pipeline runs automatically after every generation run and can also be invoked manually against any completed run directory or any arbitrary JSONL dataset. It loads the accepted records from `output.jsonl`, the rejected records from `rejected.jsonl`, and the run metadata from `run_manifest.json`, then composes five independent metric modules into a single `EvaluationReport` object. That object is serialized to `evaluation_report.json` as the canonical machine-readable form and rendered to HTML and PDF for human review. The entire accepted corpus is evaluated — the pipeline does not sample — so the report describes the dataset that ships, not a proxy for it.

The design treats the report as a primary deliverable rather than an internal diagnostic. Each metric module is independently enable-able through configuration and fails soft: if a single metric raises an exception, the pipeline logs it and continues, so a failure in one module never prevents the rest of the report from being produced. The five modules run in a fixed order — diversity, quality, compliance, yield analysis, and clustering — with an explicit memory-reclamation step between the quality and compliance stages, because several of the metrics carry allocations that scale with record count and the evaluation host is memory-constrained. The report-as-deliverable pos-

ture is what makes the methodology’s quality claims checkable by a third party rather than asserted.

7.2 The Five Metric Families

Diversity. The diversity module measures lexical variety in the accepted outputs, and its headline metric is deliberately chosen to be scale-invariant. The primary grade is the Moving-Average Type-Token Ratio (MATTR), computed over a sliding 100-token window, which does not penalize larger corpora the way a raw type-token ratio does. The module also reports the hapax ratio — the fraction of vocabulary appearing exactly once, a sensitive indicator of genuine lexical expansion — and top-20 token concentration, which exposes whether a small set of words dominates the corpus. Raw unigram, bigram, and trigram ratios are retained but explicitly relabeled as size-sensitive lexical density rather than presented as the diversity grade, because they fall mechanically as corpus size grows and would mislead a cross-size comparison. A near-duplicate rate over instructions and full per-domain breakdowns complete the module. The reason for this metric design — and the episode that forced it — is documented in §7.5.

Quality. The quality module characterizes the well-formedness of the outputs independent of their semantics. It reports the distribution of output token length and character length (mean, median, standard deviation, and the 5th and 95th percentiles), the Shannon entropy of the token-length distribution, a sentence-level repetition rate, and the fraction of outputs falling below a minimum or above a maximum length threshold. It also summarizes the distribution of the stored instruction-output cosine similarity, providing a quality-side view of the same signal the hallucination gate uses. All quality metrics are additionally broken out per domain.

Compliance. The compliance module is the one documented in detail in Section 5: residual PII leak rate measured by an independent scanner, placeholder coverage and density, GDPR-surface rate, bias-flag rate, and echo-contamination rate, each with per-domain breakdowns. Its inclusion in the standard report is what makes the compliance posture of every shipped dataset a matter of record rather than assertion.

Yield analysis. The yield module reconstructs the accept-reject economics of the run from the accepted and rejected record sets. It reports the overall yield ratio, a per-reason breakdown of rejections across the five rejection categories (hallucination, privacy, bias, GDPR, and URL violations), per-domain yield, the hallucination rate against total attempts, and a batch-level yield trend that surfaces drift across the course of a run. This module is what turns the rejection log from a discard pile into an analyzable account of why the gates fired and where.

Clustering. The clustering module performs an unsupervised semantic grouping of the accepted outputs, by default over TF-IDF vectors with k-means, selecting the cluster count automatically and reporting silhouette score, per-cluster sizes, the top interpretable terms per cluster, and — most importantly — per-cluster domain purity. The clustering exists to catch failure modes that the per-record gates cannot see: mode collapse, persona collapse, or a single domain’s vocabulary saturating the corpus. A unit-level gate can pass every record individually while the corpus as a whole has collapsed into a few semantic grooves, and only a corpus-level view exposes that.

7.3 Reconciliation: Guarding Against Silent Loss

Before any metric is reported, the pipeline reconciles three independent accounts of the run: the count of records actually present in `output.jsonl`, the count in `rejected.jsonl`, and the counts recorded in `run_manifest.json` at generation time, including the per-domain distribution. If the manifest’s accepted or rejected totals disagree with the JSONL line counts, or if the domain distribution recorded at generation time does not match the domains actually present in the delivered file, the reconciliation step flags a warning in the report. A run whose three accounts agree is marked reconciled. This check is the guard against silent data loss — a crash mid-write, a truncated file, a checkpoint that did not flush — that would otherwise let a dataset ship with fewer or different records than the run believed it produced. Reconciliation makes the integrity of the artifact itself, not merely the quality of its contents, part of the attested report.

7.4 A Worked Report

The structure is best shown on a real run. The following figures are drawn directly from the evaluation report for the second production cloud run (`run_20260602_010756`), a 1,000-target production run that accepted 1,116 records against 36 rejections, with the full evaluation completing in 14.05 seconds and the reconciliation status marked *ok* — all three counts and the domain distribution in agreement.

The yield section records an overall yield of 96.9% (Figure 7.1). The 36 rejections break down as 10 hallucination rejections, 6 bias-violation rejections, and 4 GDPR-violation rejections, with zero privacy and zero URL rejections. Per-domain yield ranges from 89.3% in transport to 99.4% in privacy and education, with a hallucination rate of 0.87% against total attempts. This is the rejection economics of the run made fully legible: a buyer can see not only that 96.9% of generation was accepted, but exactly which gates removed the remainder and in which domains.

Figure 7.1. *Yield and rejection-reason breakdown from the evaluation report for production run `run_20260602_010756`. Left: overall yield of 96.9% (1,116 accepted, 36 rejected). Right: the 36 rejections by reason — 10 hallucination, 6 bias, 4 GDPR, and zero privacy or URL. These figures are read directly from the run’s machine-readable evaluation report.*

The diversity section reports a MATTR of 0.769 — the scale-invariant grade that holds near 0.77 across all three production runs documented in this paper — against a vocabulary of 7,152 unique types, a hapax ratio of 0.304, top-20 token concentration of 0.109, and a near-duplicate rate of zero. The relabeled size-sensitive ratios sit where corpus size dictates and are not read as the diversity grade. The quality section reports a mean output length of 602 tokens with a 5th-to-95th-percentile range of 400 to 817, a token-length entropy of 3.35, and a sentence-level repetition rate of 2.4%. The compliance section reports the figures established in Section 5 — a zero residual PII leak rate from the independent scanner, 52.5% placeholder coverage at a mean density of 2.91, a 47.6% GDPR-surface rate, and a 2.06% bias-flag rate — and the clustering section auto-selected six clusters whose per-cluster domain purity ranges from 0.49 to 0.99, confirming that most clusters map cleanly onto single domains

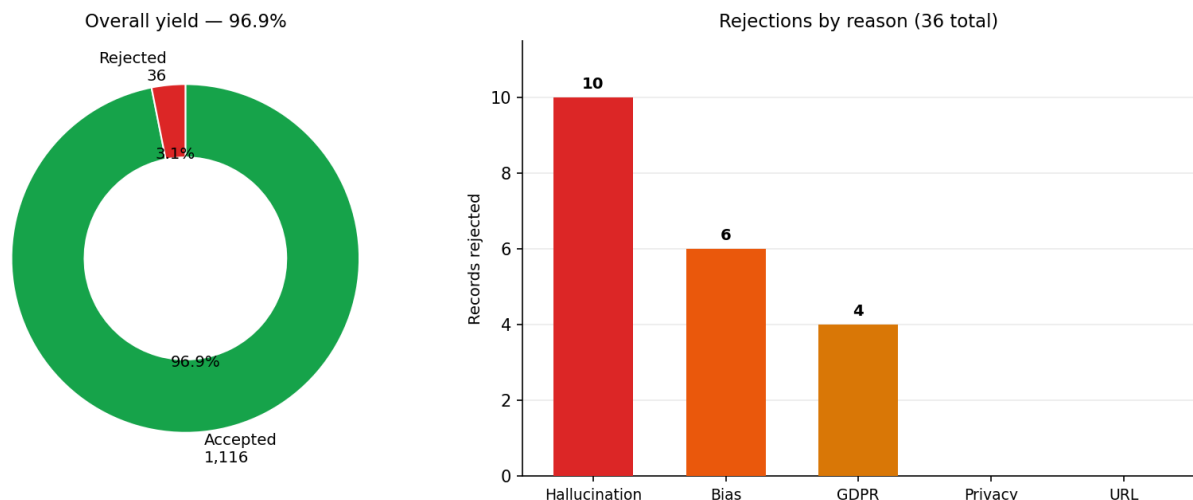


Figure 6: Figure 7.1 — Yield and rejection breakdown, run_20260602_010756

rather than collapsing across them. Every figure in this paragraph is a field in the same `evaluation_report.json` that ships with the dataset.

7.5 When the Pipeline Caught a Real Defect

The clearest demonstration that the evaluation pipeline is a working instrument rather than a decorative one is the episode in which it caught a generation defect that the per-record gates could not. During scale testing, two runs — one of 100 records and one of 250 — showed raw unigram diversity falling from 13.1% to 7.5% as the record count grew, which a naive reading would interpret as the larger dataset being lexically worse. The diversity and clustering modules together showed the real cause. The raw unigram ratio was falling for the mechanical reason that type-token ratio always falls as token mass grows; that was a metric artifact, not a quality regression. But beneath the artifact lay a genuine defect the clustering module exposed: both runs were effectively single-domain, dominated by medical billing and coding vocabulary, with the configured non-medical domains carrying zero sampling weight. The corpus had collapsed into a few semantic grooves — clusters concentrated on breach investigation, coding queries, prior authorization, and audit language — even though every individual record passed the gates.

This finding drove two methodology changes documented elsewhere in this paper. First, the diversity module was restructured so that MATTR, hapax ratio, and top-20 concentration — all scale-robust — became the reported diversity signals, with raw type-token ratios relabeled as size-sensitive density so they could never again be misread as a cross-size quality grade. Second, domain sampling was made a first-class configurable control with explicit per-domain weights, so a run could not silently collapse into one domain. The episode is the strongest evidence for the paper’s recurring claim that the measurement infrastructure is itself the product: a pipeline reaching for a single naive diversity number would have shipped a collapsed dataset and reported it as merely smaller, whereas the multi-metric, corpus-level evaluation

surfaced both the artifact and the real defect and converted them into concrete fixes. The evaluation pipeline does not merely describe the dataset; it is the mechanism by which the methodology improves.

8 Results

The preceding sections each validated one subsystem of the methodology against production data: the hallucination gate in Section 4, the compliance architecture in Section 5, the throughput and economics in Section 6, and the evaluation pipeline in Section 7. This section steps back from the subsystems and presents the three production runs as runs — a consolidated account of what was produced across both generation substrates, and the single finding that the whole paper turns on: the methodology produces equivalent datasets whether generation runs locally on SynthEval Studio or remotely on SynthEval Cloud. The data substantiate substrate-independent quality parity, which is the empirical foundation of the two-substrate thesis.

8.1 The Three Runs

Three production runs anchor the results. The local run, `run_20260521_230610`, generated 1,000 accepted records on SynthEval Studio using the RTX 4090 and the Bitsand-Bytes 4-bit model. The two cloud runs, `run_20260601_015003` and `run_20260602_010756`, generated 1,110 and 1,116 accepted records respectively on SynthEval Cloud using the Modal A10G and the AWQ-vLLM model. All three completed with their evaluation reports reconciled — the accepted and rejected counts in each run’s delivered files agree with its run manifest and domain distribution, so each run is an internally consistent artifact, not merely a set of headline numbers.

Table 8.1 consolidates the runs across the metrics the previous sections established. The figures are drawn from each run’s machine-readable evaluation report, with one deliberate exception noted below the table.

Table 8.1. *Consolidated results across the three production runs. Quality metrics (MATTR, cosine, near-duplicate rate) and yield metrics are taken from each run’s stored evaluation report. The PII leak rate is reported using the current independent compliance scanner applied uniformly to all three runs, for the reason given below the table.*

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
Substrate	Studio / RTX 4090	Cloud / A10G	Cloud / A10G
Generation format	BitsandBytes NF4	AWQ-vLLM	AWQ-vLLM
Accepted records	1,000	1,110	1,116
Overall yield	98.3%	96.4%	96.9%
MATTR (diversity)	0.764	0.768	0.769
Mean accepted cosine	0.730	0.730	0.732
Hallucination rate	0.79%	0.69%	0.87%
Near-duplicate rate	0.0%	0.0%	0.0%
Residual PII leak rate	3.8%	0.72%	0.0%

Metric	Local (0521)	Cloud R1 (0601)	Cloud R2 (0602)
Bias-flag rate	1.4%	1.8%	2.06%

The PII leak rate requires one explanation, because it is the only metric in the table not read verbatim from the stored reports. Each run’s stored report recorded the leak rate measured by the compliance scanner *as it existed when that run was generated*. The local run’s stored report records a 0.1% leak under the scanner of that era. Reporting the three runs on three different scanner versions would not be a valid comparison, so the leak rate in Table 8.1 is instead the result of applying the current, hardened independent scanner — the one documented in Section 5.6 — uniformly to all three runs. Under that single instrument the local run shows 3.8%, the first cloud run 0.72%, and the second cloud run 0.0%. The gap between the local run’s 0.1% historical figure and its 3.8% under the current scanner is itself the Section 5.6 lesson made concrete: a leak detector that shares the masker’s blind spots under-reports precisely the leaks that matter, which is why the current scanner uses deliberately broader patterns. The compliance trajectory is discussed in §8.3; the quality parity in §8.2 is unaffected by it, as the quality metrics are scanner-independent.

8.2 The Headline Finding: Quality Parity Across Substrates

The central result is that the three runs are equivalent in quality despite being generated on two different substrates with two different quantization formats. The scale-invariant diversity grade, MATTR, is 0.764 on the local run, 0.768 on the first cloud run, and 0.769 on the second — a spread of half a percentage point across substrates (Figure 8.1). The mean instruction-output cosine similarity, the fidelity signal from the hallucination gate, is 0.730, 0.730, and 0.732 — a spread of two thousandths. The near-duplicate rate is zero in all three. The hallucination rate sits between 0.69% and 0.87% across all three. By every quality measure the methodology reports, a dataset generated on the cloud A10G through AWQ-vLLM is indistinguishable from one generated on the local RTX 4090 through BitsandBytes.

Figure 8.1. *Quality parity across substrates. Left: lexical diversity (MATTR) is within half a percentage point across the local run and both cloud runs, all above the ~76% quality bar established during local development. Right: mean accepted instruction-output cosine similarity is within two thousandths across all three. The cloud substrate matches the local quality bar rather than approximating it.*

This parity is the empirical claim on which the two-substrate thesis rests. Section 6 established that the cloud substrate is roughly thirteen times faster than local at the lowest per-record cost of any format tested; that speed advantage would be worthless if it came at a quality cost. Table 8.1 and Figure 8.1 show it does not. The reason parity holds is the one given in Section 4.5: the quality-determining components — the hallucination gate and its local embedding model, the masking architecture, the evaluation pipeline — are identical across substrates and run on the same local hardware regardless of where tokens are generated. Only the generation step differs, and the quantization-format selection of Section 6.2 ensured the cloud generation format holds the same quality bar the local format set. The parity is therefore not a fortunate

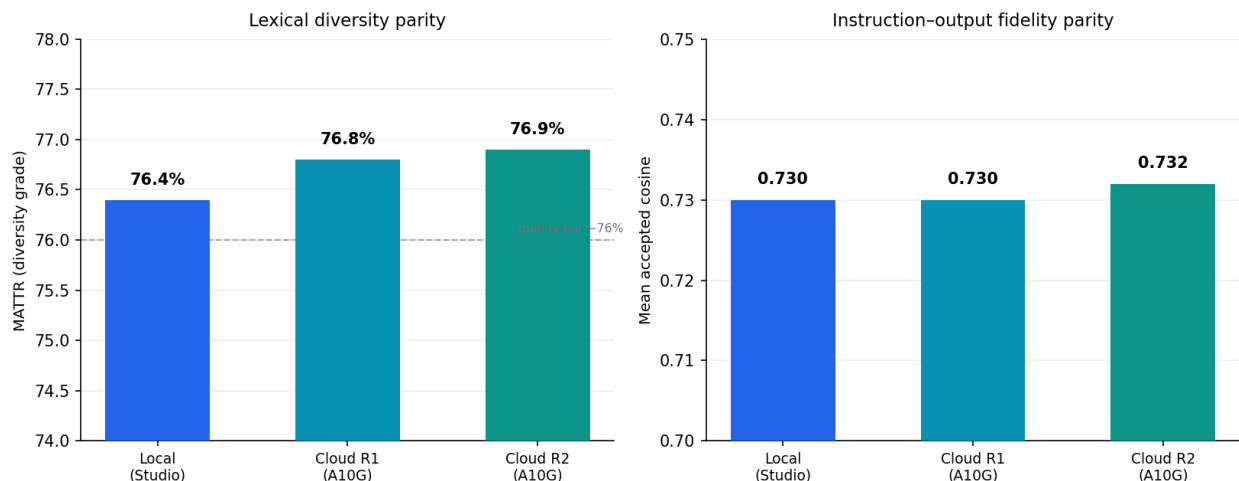


Figure 7: Figure 8.1 — Quality parity across substrates

coincidence; it is the designed consequence of holding the methodology constant and varying only the substrate beneath it.

8.3 Yield, Per-Domain Behavior, and the Compliance Trajectory

Overall yield is high and consistent: 98.3% local, 96.4% and 96.9% cloud. The modest reduction on the cloud runs is not a quality regression but a difference in rejection composition — the cloud runs rejected a few more records on bias, GDPR, and URL grounds, all of which are the compliance gates doing their work, while the hallucination rate stayed essentially flat across all three. In every run the great majority of generation was accepted, and the records that were not are accounted for in the rejection logs by reason.

The per-domain yield reveals one consistent pattern worth naming (Figure 8.2). Transport is the lowest-yielding domain on the cloud runs — 85.3% on the first cloud run against 100% locally — while education and privacy are consistently near the top across both substrates. The transport dip is concentrated in placeholder-density retries on the format-bearing transport templates rather than in hallucination or compliance rejections, and it is the clearest candidate for targeted improvement in future runs. Naming it here, rather than averaging it away, is consistent with the methodology’s posture: the per-domain breakdown exists precisely so that a domain-specific weak point is visible rather than hidden in an aggregate.

Figure 8.2. *Per-domain yield for the local run and the first cloud run. Education and privacy yield near the top on both substrates; transport is the consistent low point on the cloud substrate (85.3% versus 100% locally), concentrated in placeholder-density retries on format-bearing transport templates. The dotted line marks the cloud run’s overall yield.*

Both runs are also stable across their duration. The batch-level yield trend for every run is flat — the local run holds between 0.73 and 0.74 across its ten batches, and the first cloud run holds between 0.72 and 0.75 across its twelve — with no downward

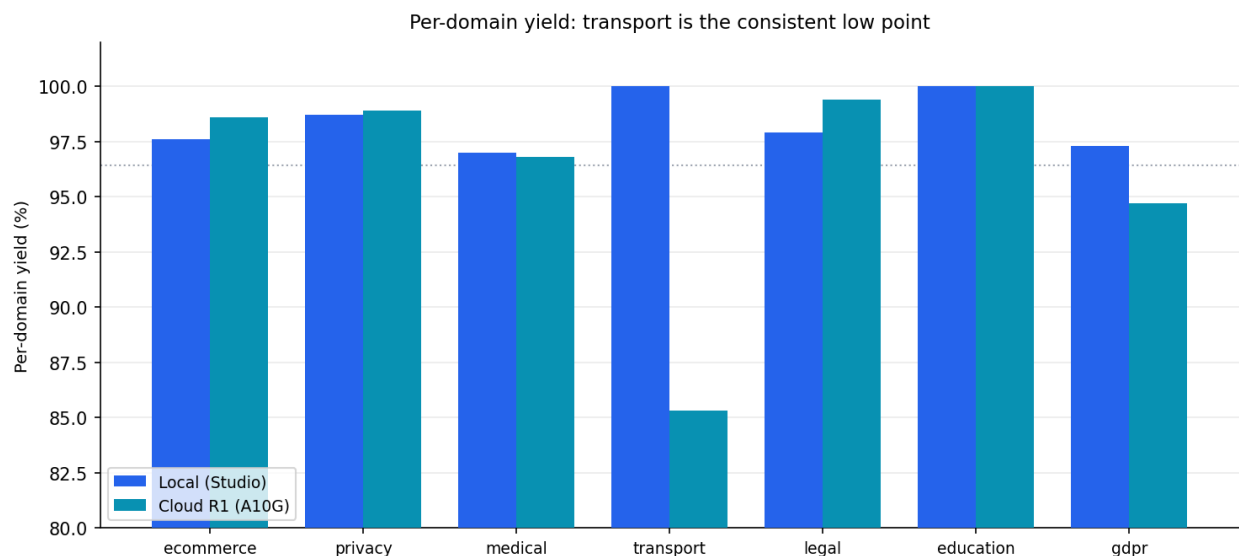


Figure 8: Figure 8.2 — Per-domain yield across runs

drift as generation proceeds. A run that degraded over its course, exhausting its usable prompt space or accumulating repetition, would show a declining batch-yield trend; none of the three runs does. The clustering analysis corroborates this: each run’s accepted corpus resolves into eleven or more semantic clusters with mostly high per-domain purity, indicating the records spread across the configured domain space rather than collapsing into a few grooves — the failure mode Section 7.5 documents the pipeline catching when it did occur.

The compliance trajectory across the three runs is the one axis where the runs are deliberately *not* at parity, and the difference is an improvement rather than a substrate effect. The local run predates the masking hardening — the span-protection and truncated-identifier detection of Section 5.2 — and under the current scanner it shows a 3.8% residual leak. The two cloud runs, generated after the hardening, show 0.72% and 0.0%. This is the before-and-after of the compliance work documented in Section 5.7, and it is why the local run, though it is the correct anchor for the hallucination-gate calibration of Section 4 and for the quality-parity finding above, is presented as a pre-hardening baseline on compliance rather than as a third parity point. The quality parity holds across all three; the compliance posture improves from the local baseline to the hardened cloud runs and reaches a scanner-verified zero residual leak in the final run.

8.4 What the Results Establish

Taken together, the three runs establish three things. First, the methodology produces datasets that meet a consistent quality bar — MATTR near 0.77, mean cosine near 0.73, zero near-duplicates — and it does so reproducibly across three independent runs. Second, that quality bar is held identically on both generation substrates, so the substrate is genuinely an operational choice rather than a quality tradeoff: the cloud substrate buys roughly thirteen times the throughput at lower per-record cost

without surrendering any measured quality. Third, the compliance architecture, once hardened, drives residual PII leak to a scanner-verified zero on the delivered dataset while the rejection logs account for everything removed. The results do not claim perfection — the transport-domain yield and the residual bias-flag rate are named, not hidden — but they substantiate the paper’s core claim: one quality-gated methodology, deployed across two complementary substrates, produces attested, compliant, equivalent datasets, and the choice of substrate is dictated by volume and timeline rather than by any difference in the data it yields.

9 Economic Implications

The measurements in the preceding sections support an economic argument that is broader than any single cost figure. This section draws the implications out: why the near-zero marginal cost of local generation is decisive during development, how the choice between substrates should be made for a given production engagement, why an owned-infrastructure cost structure differs in kind rather than degree from per-token API rental, and what the attested-deliverable methodology is worth above raw generation. The argument is analytical and treats frontier generation APIs throughout as a neutral cost-and-capability baseline, not an adversary.

9.1 The Development Economics: Converging on Quality, Not Budget

The most consequential economic property of local generation is not that it is cheap in production but that it is effectively free during development, and that this changes what kind of engineering becomes possible. On amortized local hardware the marginal cost of a generation run is electricity — on the order of thirteen cents per thousand records, as Section 2.3 establishes — which means the cost of an additional development iteration is, for practical purposes, zero. The engineer is therefore free to iterate until the quality metrics converge, rather than until a budget is exhausted.

The development history documented in this paper is the direct evidence. The hallucination threshold was recalibrated from an early permissive value to its production setting by examining the empirical similarity distribution across repeated runs (Section 4.3). The masking architecture’s span-protection and truncated-identifier detection were developed and validated against a measured leak rate that had to be driven from a multiple-percent residual to zero (Section 5.7). The quantization-format selection that underpins the cloud substrate required standing up three formats, tuning the weakest across four iterations before concluding its quality ceiling was structural, and measuring all three end-to-end through the evaluation pipeline (Section 6.2). The diversity-metric redesign followed from running the pipeline at multiple scales and observing a domain-collapse defect (Section 7.5). Each of these is an iterative, measurement-driven investigation, and each consumed many full generation-and-evaluation cycles.

The economic point is what those cycles would have cost under per-token API pricing. At the frontier and mid-tier rates documented in Section 2.3, and the token profile measured there, a single thousand-record generation costs several dollars; the development arc above consumed far more than a single run across its calibration, hard-

ening, format-selection, and scale-testing phases. Conducted against a metered API, that same body of work would have run into the thousands of dollars before the first production-quality dataset shipped — and, more importantly, every iteration would have carried a visible price tag that creates pressure to stop early. The structural consequence is that metered development biases toward shipping the first adequate dataset, while amortized local development permits shipping the dataset that the quality metrics, not the budget, declare finished. The two produce different products: one sized to the cost of generating it, the other sized to the standard the publisher chose to enforce. Local generation is, in this sense, less a production substrate than the substrate on which a defensible methodology can be developed at all.

9.2 The Production Decision Framework

Once the methodology is fixed, the choice of production substrate is an operational decision governed by two variables: the volume of the deliverable and the timeline it must meet. The quality of the output does not enter the decision, because Section 8 established that quality is held constant across substrates. This is what makes the choice a clean engineering tradeoff rather than a compromise.

The governing quantities are established in earlier sections. Local generation runs at approximately 251 accepted records per hour at roughly thirteen cents per thousand records in electricity. Cloud generation on the A10G runs at approximately 3,300 records per hour — about thirteen times faster — at roughly forty-five cents per thousand records all-in. The decision rule that follows is straightforward. For low-volume deliverables, or deliverables on flexible timelines, local generation is the natural substrate: its marginal cost is the lowest available and its throughput is more than adequate when wall-clock time is not the binding constraint. For high-volume deliverables under fixed deadlines, the local substrate’s throughput becomes the binding constraint — a 20,000-record deliverable takes on the order of five days locally — and the cloud substrate is the natural choice, trading a still-negligible absolute cost increase for an order-of-magnitude reduction in delivery time.

The governing quantities are established in earlier sections. Local generation runs at approximately 251 accepted records per hour at roughly thirteen cents per thousand records in electricity. Cloud generation on the A10G runs at approximately 3,300 records per hour — about thirteen times faster — at roughly forty-five cents per thousand records all-in. The decision rule that follows is straightforward. For low-volume deliverables, or deliverables on flexible timelines, local generation is the natural substrate: its marginal cost is the lowest available and its throughput is more than adequate when wall-clock time is not the binding constraint. For high-volume deliverables under fixed deadlines, the local substrate’s throughput becomes the binding constraint — a 20,000-record deliverable takes on the order of five days locally — and the cloud substrate is the natural choice, trading a still-negligible absolute cost increase for an order-of-magnitude reduction in delivery time. The same operational constraints extend to the choice of cloud GPU itself, because SynthEval Cloud selects its Modal GPU through a single configuration value, the rented tier — A10G, A100, or H100 — can be matched to the deliverable’s volume and deadline without any change to the methodology, so the operator scales the rented hardware up for a larger or more urgent engagement and back down for a routine one, paying only for

the tier each engagement requires.

The crossover is therefore not a cost crossover — local is cheaper per record at every volume — but a time crossover. The relevant question for any engagement is whether the deliverable’s deadline can be met at the local throughput; if it can, local is preferred on cost, and if it cannot, cloud is preferred on time at a cost that remains a small fraction of the engagement’s value. And because the cloud substrate scales on two further axes — vertically to larger GPU tiers and horizontally across concurrent containers, both documented in Sections 2.3.1 and 6 — the timeline that the cloud substrate can meet is itself adjustable by provisioning, so that even very large deliverables under tight deadlines remain within reach without changing the methodology. The substrate decision is thus reducible to a single operational question per engagement, with quality removed from the equation entirely.

9.3 Owned Infrastructure Versus Metered Rental

The cost comparison in Section 2.3 is most usefully read not as a numerical ratio but as a difference in cost *structure*. Per-token API generation imposes a linear variable cost that scales without limit: every record, on every run, on every iteration, incurs a charge proportional to the tokens it consumes, and that charge recurs identically each time the work is repeated. Owned generation infrastructure imposes a fixed capital cost — a consumer GPU for the local substrate, a serverless GPU account for the cloud substrate — that amortizes across an unbounded number of subsequent runs, leaving only a small marginal cost per run that is electricity locally and metered GPU-seconds in the cloud.

These are not two points on the same cost curve; they are two different curves. The implication compounds over the lifetime of a synthetic-data operation. A publisher generating datasets repeatedly — across domains, across client engagements, across the development-and-regeneration cycles that any evolving methodology requires — pays the metered cost every time under the rental model, and pays it once, amortized, under the owned model. The per-record figures in Section 2.3 show local generation at roughly 26 to 52 times the cost efficiency of mid-tier and frontier APIs respectively at a single point in time; over the repeated use that characterizes a real operation, the gap widens, because the fixed cost is paid once while the metered cost is paid on every repetition. The owned-infrastructure structure is what converts synthetic-data generation from a recurring operating expense into a largely amortized capital one — and it is available, through the cloud substrate, even to a publisher who does not wish to operate local hardware, because the cloud substrate’s per-record cost still reflects metered GPU-seconds rather than per-token model rental.

This structural difference is stated as an economic fact rather than a competitive claim. Per-token APIs price access to a frontier model with no infrastructure to operate, which is a real and reasonable value for many uses; the point here is narrowly that for the repeated, iterative, high-volume generation that a synthetic-data methodology entails, a fixed-cost-plus-low-marginal structure and a purely linear structure produce materially different total costs, and the methodology documented in this paper is built on the former.

9.4 The Value of Attestation

The economic argument to this point concerns the cost of generating tokens. The larger economic implication concerns what is generated. A raw generation API, on either a rental or an owned substrate, produces tokens; the methodology documented in this paper produces an attested deliverable — a dataset accompanied by a machine-readable evaluation report covering rejection reasons, scale-invariant diversity, placeholder coverage, an independently audited residual PII leak rate, and a reconciliation of the run’s own counts. The difference between those two outputs is the entire production layer documented in Sections 4 through 7.

The economic value of that layer is realized at the point of use, in a regulated commercial setting. A buyer fine-tuning a model for clinical, legal, or financial application cannot use raw generated tokens directly: the tokens must still be filtered for instruction-output drift, masked for personal data against the relevant regulatory framework, evaluated for diversity and contamination, and audited for residual leakage before the dataset can be defensibly deployed. Those steps are not optional for a regulated buyer; they are the difference between a dataset that can be used and one that cannot. A methodology that performs them at generation time and ships the evidence alongside the data has therefore done work the buyer would otherwise have to do — or, more commonly, would have to leave undone and absorb as risk. The legal and regulatory exposures catalogued in Sections 2.1 and 2.2 are the measure of that risk, and an attested, provenance-documented, compliance-audited synthetic dataset is the artifact that removes it.

The economic implication is therefore that the methodology’s value is not primarily the cost saving on token generation, substantial as that is, but the cost and risk it removes downstream. The token-generation economics determine whether a defensible dataset can be developed and produced affordably; the attestation layer determines whether the resulting dataset is worth more to a regulated buyer than the raw tokens that underlie it. The two substrates make the first true on either a local or a cloud footing; the production layer, identical across both, makes the second true regardless of which substrate generated the data. That is the economic shape of the methodology: cheap to develop and produce because the substrate is owned, and valuable to receive because the output is attested.

10 Limitations

A methodology that asks regulated buyers to trust its output earns that trust partly by stating plainly what it does not and cannot guarantee. This section names the limitations of the work honestly, each followed where applicable by its mitigation or the path to addressing it, without overstating either the problem or the remedy. The limitations fall into four groups: the bounds of pattern-based compliance, the bounds of the calibrated thresholds, the bounds of the substrates, and the bounds of the evidence presented in this paper.

10.1 Pattern-Based Compliance Cannot Be Proven Complete

The masking and leak-detection layers are pattern-based, operating on free-form model output with regular expressions and keyword sets. This imposes a hard epistemic limit: pattern matching cannot be proven complete. The methodology earned a measured zero residual PII leak rate on its final production run, corroborated by an independent audit scanner using deliberately broader patterns than the masker, as Section 5 documents — but as the audit script itself acknowledges, that is strong evidence, not mathematical proof. A novel output format can surface a personal-data shape that no existing pattern anticipates. The development history makes this concrete rather than hypothetical: three distinct IP-address-shape edge cases were encountered and resolved across iterations before the detection logic converged, and the honest reading of that history is that a fourth, as-yet-unseen shape could exist in a format the pipeline has not yet generated.

The honest claim is therefore bounded precisely: the residual PII leak rate is *measured and independently audited at zero, never guaranteed at zero*. The mitigation is structural rather than absolute — the independent scanner is deliberately broadened to over-report rather than under-report, so that the failure mode biases toward flagging a clean record for review rather than passing a leaked one, and the always-mask floor for the highest-sensitivity identifier types is domain-independent. These measures make an undetected leak less likely; they cannot make it impossible. A buyer in a regulated setting should read the zero leak rate as a rigorously measured and independently corroborated result, with the residual epistemic risk that attaches to any pattern-based system on open-ended input.

10.2 Threshold Calibration Is Empirical, Not Adaptive

The hallucination thresholds of 0.55 for general domains and 0.58 for legal and medical were calibrated once, against the embedding model’s behavior on this pipeline’s instruction distribution, and applied uniformly across all production runs. They do not self-adjust. A new domain with a different vocabulary profile, or a change of embedding model, or a change of generation model, would shift the underlying similarity distribution and could render the existing thresholds miscalibrated — too permissive in one direction or too aggressive in the other.

This is a real constraint on portability, but a bounded and easily addressed one. Recalibration is not automated, but it is straightforward to determine: a 100-record run through the evaluation pipeline surfaces the empirical similarity distribution for the new configuration, from which a corrected threshold is read directly, and the threshold is then set through a single configuration value with no code change, exactly as the per-domain thresholds are set today. The limitation is therefore that calibration is a manual step that must be repeated when the embedding model, generation model, or domain set changes; the mitigation is that the step is cheap, fast, and configuration-driven rather than a development effort.

10.3 Substrate Bounds

The local substrate is bounded by a single GPU. SynthEval Studio runs on one RTX 4090, and its throughput is fixed by that hardware: approximately 251 accepted records per hour, with no horizontal scaling available locally. A larger local workload does not run faster; it runs longer. This is the structural limit that motivates the cloud substrate in the first place, and within the local substrate it is not removable — throughput is capped by the single owned card. The mitigation is the cloud substrate itself, which is the subject of much of this paper; the limitation is simply that local generation cannot scale beyond what one GPU delivers, and any deliverable whose timeline exceeds the local throughput must move to the cloud substrate rather than being accelerated locally.

The cloud substrate’s own scaling figures carry a different and important limitation: several are projections, not measurements. The A10G throughput and cost are measured from production runs, but the A100 and H100 GPU-tier figures, and the horizontal container-concurrency figures, are conservative projections derived from the measured compute-bound behavior of the A10G and from published GPU specifications and pricing, as Sections 2.3.1 and 6 state explicitly. They are presented as projections and should be read as such: the single-container A10G economics are demonstrated, while the larger-GPU and multi-container figures are reasoned estimates pending direct measurement. The horizontal-scaling projection additionally assumes the local evaluation stage is parallelized to keep pace.

10.4 The Evaluation-Layer Bias Metric Over-Reports in Regulated Domains

The bias-flag rate reported on the accepted set — between 1.4% and 2.06% across the production runs — requires careful interpretation, because the bias metric is pattern-based and produces expected false positives in precisely the domains the methodology targets. The evaluation layer’s bias detector flags language referencing protected attributes and discrimination. In domains such as GDPR, legal, and privacy, that language is frequently the instructed, correct content rather than a defect: a faithful GDPR record addressing special-category data must name the categories the regulation enumerates — racial or ethnic origin, health, biometric data — and a legal memo analyzing disparate impact must discuss protected classes by name. The measured GDPR-surface rate approaching unity in the GDPR domain reflects exactly this — the domain’s correct subject matter is regulatory language.

The consequence is that the accepted-set bias-flag rate is concentrated in these regulated domains and is better understood as the metric correctly detecting regulated subject matter than as residual harmful bias in the dataset. The flag’s verdict reflects the metric’s deliberate conservatism, not a defect in the records it flags. This is a limitation of the metric’s interpretability rather than of the data: a reader cannot take the bias-flag rate at face value as a measure of harmful content, and must instead read it in the context of which domains generated the flags. The methodology reports the rate transparently rather than suppressing it — consistent with the gate-passed-not-provably-perfect posture established in Section 5 — but the honest interpretation of the number is that it largely measures the presence of the regulated terminology the

dataset was commissioned to contain.

10.5 The Bounds of the Evidence

Two limitations attach to the evidence in this paper rather than to the methodology itself. First, the production runs that anchor the results are on the order of one thousand records each. The methodology’s behavior at the ten-thousand to one-million-record scale that the economic projections describe is itself projected from the thousand-record runs and the per-record cost structure, not yet demonstrated at that scale; the local evaluation pipeline’s memory profile in particular, which scales super-linearly with record count, is a known pressure point at large scale that has not been exercised in production. Second, the compliance comparison across runs is not a clean three-way parity: the local run predates the masking hardening and is presented as a pre-hardening baseline rather than a third equivalent point, so the demonstrated compliance result rests on the two cloud runs.

These bounds do not undercut the paper’s central claims — quality parity across substrates, the dual-signal gate’s discrimination, the independently audited zero leak rate, and the cost-and-throughput economics are all established on measured data — but they delimit them. The methodology is demonstrated at the thousand-record scale and projected beyond it; the compliance result is demonstrated on the hardened cloud runs; and the larger-substrate economics are projected from measured single-container behavior. Stating these bounds is itself part of the methodology’s posture: the same transparency that reports a residual bias-flag rather than claiming an unqualified zero applies to the scope of the evidence, so that what is measured and what is projected are never conflated.

11 Appendix

This appendix provides reference material for readers verifying or reproducing the work: the operative configuration values, an inventory of the runs cited in the paper, the exact environment specification for both substrates, and a glossary of the technical terms used throughout. All values are drawn from the production configuration and source as of the runs documented in Section 8.

11.1 Configuration Reference

The methodology is configuration-driven; the values below are the operative settings for the production cloud runs. The same structure governs the local substrate, differing only in the generation backend.

Table 11.1. *Hallucination gate. The general threshold applies to all domains except the two listed.*

Parameter	Value
Default threshold	0.55
Legal threshold	0.58

Parameter	Value
Medical threshold	0.58
Borderline margin	± 0.05 cosine of active threshold
Embedding batch size	22

Table 11.2. *Generation parameters.*

Parameter	Value
Generation batch size	128
Temperature	0.3
Top-p	0.92
Repetition penalty	1.1
Max new tokens	1024
Over-generation ratio	0.25

Table 11.3. *Active domains and sampling weights (sum to 1.0).*

Domain	Weight
ecommerce	0.14
privacy	0.14
medical	0.14
transport	0.14
legal	0.14
education	0.15
gdpr	0.15

Table 11.4. *Masking and placeholder control.*

Parameter	Value
Active masking profile	gdpr
PII substitution	enabled
Placeholder mode	balanced
Target placeholders per record	2.5
Minimum placeholders per record	1
Maximum placeholders per record	6

11.2 Run Inventory

Table 11.5. *Runs referenced in this paper. Production runs anchor the results in Section 8; comparison runs support the model- and format-selection analyses in Section 6. MATTR is omitted for the Q8 run because that run predates the diversity module in the evaluation pipeline.*

Substrate / GPU	Model & format	Records	Role in paper
run_20260521_230610 Studio / RTX 4090	Mistral-Nemo, BitsandBytes NF4	1,000	Local production; §4 calibration anchor, §8 quality anchor, §5 pre-hardening compliance baseline
run_20260601_015003 Cloud / A10G	Mistral-Nemo, AWQ-vLLM	1,110	Cloud production (R1); §8 results, §5/§7 validation
run_20260602_010756 Cloud / A10G	Mistral-Nemo, AWQ-vLLM	1,116	Cloud production (R2); §7 worked report, §8 results, scanner-verified zero leak
run_20260523_021522 Cloud / A10G	Mistral-Nemo, 8-bit (Q8)	100	Quantization comparison point (§6)
GGUF runs R1-R4 A10G	Mistral-Nemo, GGUF Q4_K_M	—	Quantization-format A/B (§6.2/§6.3)
run_20260525_011036 A10G	Mistral-Nemo, BitsandBytes NF4	—	Quantization-format A/B (§6.2/§6.3)
Qwen-2.5-32B (F16) A100-40GB	Qwen-2.5-32B-Instruct, F16	—	Model-selection comparison (§6.2)
Qwen-2.5-32B (NF4) A100-40GB	Qwen-2.5-32B-Instruct, BnB NF4	—	Model-selection comparison (§6.2)
Qwen-2.5-32B (NF4 pre) A100-40GB	Qwen-2.5-32B-Instruct, pre-quant NF4	—	Model-selection comparison (§6.2)

11.3 Environment Specification — Local Substrate (SynthEval Studio)

Table 11.6. *Local orchestration, embedding, and evaluation environment. The generation and embedding models are quantized in-house. Process isolation via `pytest --forked` is required for the test suite, for the reason documented in Section 3.11.*

Component	Specification
GPU	NVIDIA RTX 4090 (24 GB)
Host environment	WSL2
PyTorch	2.4.1 + cu121
Transformers	≥ 4.40 , < 5.0
sentence-transformers	$\geq 2.7.0$

Component	Specification
Embedding model	Snowflake Arctic Embed L v2.0, 4-bit (in-house)
Generation model	Mistral-Nemo-Instruct-2407, 4-bit BitsandBytes (in-house)
SciPy / scikit-learn	1.11.4 / 1.3.2
PII detection & masking	Hand-written compiled-regex pattern registry (Python re)
Report generation	WeasyPrint $\geq$ 62.0, Jinja2 $\geq$ 3.1.4
Test isolation	pytest $\geq$ 8.2.0, run with <code>--forked</code>

11.4 Environment Specification — Cloud Substrate (SynthEval Cloud)

Table 11.7. *Remote Modal vLLM serving environment. The container environment is isolated from the local environment and pins its own PyTorch and vLLM versions. Evaluation and embedding remain on the local substrate (Table 11.6).*

Component	Specification
Modal app name	syntheval-cloud-vllm
Serving class	MistralNemoAWQVllm
GPU	NVIDIA A10G (24 GB)
Container Python	3.11 (Debian slim image)
vLLM	0.8.5.post1
PyTorch (container)	2.6.0
Transformers (container)	4.51.3
Tokenizers	0.21.1
Modal client (local)	$\geq$ 0.64.0
Model	Mistral-Nemo-Instruct-2407 AWQ (asymmetric W4A16, in-house)
Quantization scheme	compressed-tensors
dtype	float16
max_model_len	2560
gpu_memory_utilization	0.90
tensor_parallel_size	1
Max containers	1 (current production; the horizontal-scaling knob of §6)
Scaledown window	300 s (warm-hold after last call)
Per-call timeout	600 s

The AWQ asymmetric W4A16 model pins vLLM 0.8.5.post1 specifically: its scheme matcher accepts asymmetric weights where earlier versions required symmetric quantization. The `max_containers` value of 1 is the current production setting; raising it is the horizontal-scaling axis projected in Sections 2.3.1 and 6.

11.5 Glossary

Table 11.8. *Technical terms used in this paper.*

Term	Definition
AWQ	Activation-aware Weight Quantization. A 4-bit quantization method that preserves the weights most important to activations, used for the cloud generation model.
BitsandBytes NF4	A 4-bit NormalFloat quantization format used for the local generation model and as a cloud comparison point.
CLS pooling	Taking the embedding of the special classification token as the representation of an entire input sequence; how the embedding model produces its instruction and output vectors.
Compressed-tensors	The quantization format identifier under which vLLM loads the AWQ model.
Cosine similarity	A measure of semantic correspondence between two embedding vectors, ranging from -1 to 1 ; the primary signal of the hallucination gate.
GGUF Q4_K_M	A 4-bit k-quant format served via llama-cpp; evaluated and rejected on quality grounds in Section 6.
Hapax ratio	The fraction of a vocabulary appearing exactly once; a sensitive indicator of genuine lexical expansion.
KV cache	The key-value cache holding attention state during generation; its size bounds how many sequences can be served concurrently.
MATTR	Moving-Average Type-Token Ratio; a scale-invariant measure of lexical diversity computed over a sliding window, used as the headline diversity grade.
NF4	NormalFloat 4-bit; the data type used by BitsandBytes quantization.
Paged attention	The vLLM memory-management technique that pages the KV cache to enable efficient continuous batching.
Placeholder density	The mean number of typed placeholders per record after masking.
Type-token ratio (TTR)	The ratio of unique words to total words; size-sensitive, and therefore relabeled as lexical density rather than used as a cross-size diversity grade.
W4A16	4-bit weights, 16-bit activations; the precision profile of the AWQ cloud model. The asymmetric variant requires vLLM 0.8.5.post1.
Yield	The proportion of generated records accepted after the full quality and compliance cascade.

12 References

Adler, B., Agarwal, N., Aithal, A., Anh, D. H., Bhattacharya, P., Brundyn, A., Casper, J., Catanzaro, B., Clay, S., Cohen, J., Das, S., Dattagupta, A., Delalleau, O., Derczynski, L., Dong, Y., Egert, D., Evans, E., Ficek, A., Fridman, D., . . . You, J. (2024). *Nemotron-4 340B technical report* (arXiv:2406.11704). arXiv. <https://arxiv.org/abs/2406.11704>

AI Lawsuit Tracker. (2026, April). *AI lawsuit tracker (2026): OpenAI, NYT & 164 other AI copyright litigation cases*. <https://ailawsuittracker.com/>

Anthropic. (2026, January 22). Claude’s new constitution. <https://www.anthropic.com/news/claude-new-constitution>

Anthropic. (2026, May). *Pricing — Claude API docs*. Anthropic. <https://platform.claude.com/docs/claude/pricing>

Authors Alliance. (2026, May 14). *Bartz v. Anthropic fairness hearing: Final reminder, 91.3% claims rate, and updates from the docket*. <https://www.authorsalliance.org/2026/05/bartz-v-anthropic-fairness-hearing-final-reminder-91-3-claims-rate-and-updates-from-the-docket/>

Authors Guild. (2025, September 26). *Bartz v. Anthropic Settlement: What Authors Need to Know*. <https://authorsguild.org/advocacy/artificial-intelligence/what-authors-need-to-know-about-the-anthropic-settlement/>

Berberich, M. (2025, June 13). *AI and data protection: Using AI systems in compliance with the GDPR*. SRD Rechtsanwälte. <https://www.srd-rechtsanwaelte.de/en/blog/privacy-ai-gdpr>

CloudZero. (2026a, May). *Anthropic Claude API pricing in 2026: Every model, token rate, and cost lever*. <https://www.cloudzero.com/blog/claude-api-pricing/>

DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning* [Model release and technical documentation]. GitHub. <https://github.com/deepseek-ai/DeepSeek-R1>

European Parliament & Council of the European Union. (2016). *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 1–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

Gemini Team, Google. (2025, June 16). *Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities*. Google DeepMind. https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf

Guan, M. Y., et al. (2024, December 20). *Deliberative alignment: Reasoning enables safer language models*. OpenAI. <https://openai.com/index/deliberative-alignment/>

International Association of Privacy Professionals. (2025, November 5). *How to train AI lawfully?* <https://iapp.org/news/a/how-to-train-ai-lawfully->

Latham & Watkins. (2025, November 13). *Getty Images v. Stability AI: English High Court rejects secondary copyright claim*. <https://www.lw.com/en/insights/getty-images-v-stability-ai-english-high-court-rejects-secondary-copyright-claim>

Mayer Brown. (2025, November 20). *Getty Images v Stability AI: What the High Court's decision means for rights-holders and AI developers*. <https://www.mayerbrown.com/en/insights/getty-images-v-stability-ai-what-the-high-courts-decision-means-for-rights-holders-and-ai-developers>

Meta AI. (2024, July 23). *Introducing Llama 3.1: Our most capable models to date*. <https://ai.meta.com/blog/meta-llama-3-1/>

Mistral AI. (2024, July 18). *Mistral NeMo*. <https://mistral.ai/news/mistral-nemo>

Mistral AI. (2025). *Mistral-Nemo-Instruct-2407* [Model card]. Hugging Face. <https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407>

Modal Labs. (2026). Modal: Serverless GPU compute platform [Computer software documentation]. <https://modal.com/docs>

National Law Review. (2026, January). *OpenAI loses privacy gambit: 20 million Chat-GPT logs likely headed to copyright plaintiffs*. <https://natlawreview.com/article/openai-loses-privacy-gambit-20-million-chatgpt-logs-likely-headed-copyright>

Norton Rose Fulbright. (2025). *Recent regulatory developments in training artificial intelligence (AI) models under the GDPR*. <https://www.nortonrosefulbright.com/en/knowledge/publications/regulatory-developments-in-training-artificial-intelligence-ai-models-under-the-gdpr>

Norton Rose Fulbright. (2026, March 13). *AI in litigation series: An update on AI copyright cases in 2026*. <https://www.nortonrosefulbright.com/en/knowledge/publications/ce8eaa55-in-litigation-series-an-update-on-ai-copyright-cases-in-2026>

Oklahoma Gas & Electric. (2026). *Residential electric service billing statement, account 13XXXXXX0-8, service period 4 April 2026 to 5 May 2026* [Utility bill, confidential primary source on file with the author].

OpenAI. (2026, May). *OpenAI API pricing*. OpenAI. <https://openai.com/api/pricing/>

Orrick, Herrington & Sutcliffe. (2025, March). *The European Data Protection Board shares opinion on how to use AI in compliance with GDPR*. <https://www.orrick.com/en/Insights/2025-European-Data-Protection-Board-Shares-Opinion-on-How-to-Use-AI-in-Compliance-with-GDPR>

PBH Applied Systems LLC. (2026a). SynthEval Studio production run telemetry: Reference run_20260521_230610 [Run manifest, evaluation report, and GPU power telemetry]. Unpublished raw data.

PBH Applied Systems LLC. (2026b). SynthEval Studio batch-size A/B testing summary and GPU telemetry capture, 22 May 2026 [gpu_telemetry_20260522_155824.csv and syntheval.log]. Unpublished raw data.

PBH Applied Systems LLC. (2026c). SynthEval Cloud technical report: Production validation of two 1,000-record runs [Internal technical report]. Unpublished manuscript.

PBH Applied Systems LLC. (2026d). SynthEval Cloud engineering reference: Architecture, batch-throughput optimization, and test validation [Internal engineering reference]. Unpublished manuscript.

PPC Land. (2026, March 29). *GDPR's AI training legal battle: Regulators converge but still clash*. <https://ppc.land/gdprs-ai-training-legal-battle-regulators-converge-but-still-clash/>

Ropes & Gray. (2026, January 14). *Getty Image loses copyright infringement claim against StabilityAI in UK's first-of-its-kind ruling*. <https://www.ropesgray.com/en/insights/viewpoint/image-loses-copyright-infringement-claim-against-stability-ai-in-uks-first>

Silicon Data. (2026, January). *Understanding LLM cost per token: A 2026 practical guide*. <https://www.silicondata.com/blog/llm-cost-per-token>

Skadden, Arps, Slate, Meagher & Flom. (2025, June 25). *CNIL clarifies GDPR basis for AI training — but it's just one part of the compliance picture*. <https://www.skadden.com/insights/publications/2025/06/cnil-clarifies-gdpr-basis-for-ai-training>

Snowflake. (2024, December 4). *Snowflake's Arctic Embed 2.0 Goes Multilingual: Empowering Global-Scale Retrieval with Inference Efficiency and High-Quality Retrieval*. Snowflake Engineering Blog. <https://www.snowflake.com/en/engineering-blog/snowflake-arctic-embed-2-multilingual/>

vLLM Project. (2026). *vLLM: Easy, fast, and cheap LLM serving for everyone* [Computer software documentation]. <https://docs.vllm.ai>

Wiggers, K. (2024, December 24). *The promise and perils of synthetic data*. TechCrunch. <https://techcrunch.com/2024/12/24/the-promise-and-perils-of-synthetic-data/>

Wilson Sonsini Goodrich & Rosati. (2026, March 31). *OpenAI prevails in landmark Italian AI and GDPR enforcement case*. <https://www.wsgr.com/en/insights/openai-prevails-in-landmark-italian-ai-and-gdpr-enforcement-case.html>

Wolters Kluwer. (2025, November 10). *The Bartz v. Anthropic settlement: Understanding America's largest copyright settlement*. Kluwer Copyright Blog. <https://legalblogs.wolterskluwer.com/copyright-blog/the-bartz-v-anthropic-settlement-understanding-americas-largest-copyright-settlement/>

Yu, P., Merrick, L., Nuti, G., & Campos, D. (2024). *Arctic-Embed 2.0: Multilingual retrieval without compromise* (arXiv:2412.04506). arXiv. <https://arxiv.org/abs/2412.04506>